

SoK: Decentralized Finance (DeFi) Attacks

Liyi Zhou^{* **}, Xihan Xiong^{*}, Jens Ernstberger^{† **}, Stefanos Chaliasos^{*}, Zhipeng Wang^{*},
Ye Wang[‡], Kaihua Qin^{* **}, Roger Wattenhofer[§], Dawn Song^{¶ **}, and Arthur Gervais^{|| **}

^{*}Imperial College London, [†]Technical University of Munich, [‡]University of Macau,

[§]ETH Zurich, [¶]University of California, Berkeley, ^{||}University College London,

^{**}Berkeley Center for Responsible, Decentralized Intelligence (RDI)

Abstract—Within just four years, the blockchain-based Decentralized Finance (DeFi) ecosystem has accumulated a peak total value locked (TVL) of more than 253 billion USD. This surge in DeFi’s popularity has, unfortunately, been accompanied by many impactful incidents. According to our data, users, liquidity providers, speculators, and protocol operators suffered a total loss of at least 3.24 billion USD from Apr 30, 2018 to Apr 30, 2022. Given the blockchain’s transparency and increasing incident frequency, two questions arise: How can we systematically measure, evaluate, and compare DeFi incidents? How can we learn from past attacks to strengthen DeFi security?

In this paper, we introduce a *common reference frame* to systematically evaluate and compare *DeFi incidents*, including both attacks and accidents. We investigate 77 academic papers, 30 audit reports, and 181 real-world incidents. Our data reveals several gaps between academia and the practitioners’ community. For example, few academic papers address “price oracle attacks” and “permissionless interactions”, while our data suggests that they are the two most frequent incident types (15% and 10.5% correspondingly). We also investigate potential defenses, and find that: (i) 103 (56%) of the attacks are not executed atomically, granting a rescue time frame for defenders; (ii) bytecode similarity analysis can at least detect 31 vulnerable/23 adversarial contracts; and (iii) 33 (15.3%) of the adversaries leak potentially identifiable information by interacting with centralized exchanges.

I. INTRODUCTION

Blockchain-based Decentralized Finance (DeFi) ecosystem has attracted a surge in popularity since the beginning of 2020. The peak total value locked (TVL) for DeFi surpassed 253 billion USD on Dec 2, 2021, with Ethereum (145 billion, 57% TVL) and BNB Smart Chain (19.8 billion, 8% TVL) sharing the majority of DeFi’s activity [1]. While DeFi certainly provides many protocols inspired by traditional finance such as cryptocurrency exchanges [2]–[4], lending platforms [5], [6], and derivatives [7], novel constructs known as flash loans [8] and atomic composable DeFi trading [9] emerged. Unfortunately, these very intertwined DeFi systems, coupled with the already well-studied vulnerability-prone smart contracts [10]–[18], broadened the threat surface of DeFi protocols. We identify that from Apr 30, 2018 to Apr 30, 2022, so-called “DeFi incidents” have accumulated to a total loss of 3.24 billion USD. Particularly exciting to interdisciplinary scholars, these harmful incidents cover a wide variety of system layers, including the network, consensus, smart contract and DeFi protocol, as well as external auxiliary services such as off-chain oracles, cross-chain bridges, centralized exchanges

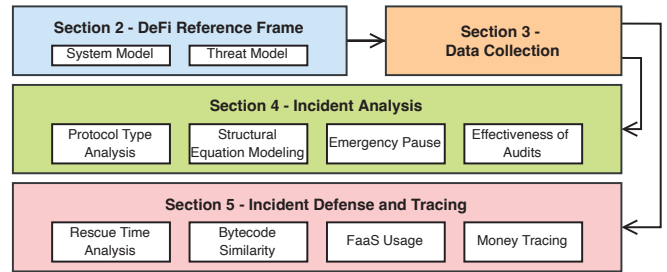


Fig. 1: Section II presents a DeFi reference frame, with a five layer system and threat model overview, allowing to categorize real-world incidents, academic works, and audit reports (cf. Section III). Section IV studies the collected DeFi incidents with statistical analysis. Section V shows how to identify adversarial and victim contracts, how to front-run adversaries, and how to trace adversarial funds. The paper concludes with a discussion in VI, related works in VII and a closure in VIII.

etc. Understanding DeFi incidents hence requires a vertical understanding of all relevant system layers and architectures.

For the first time in history, the information security community has access to a transparent, broad, timestamped, and non-repudiable dataset of million-dollar security-related incidents. In this work, we leverage the blockchain as an open dataset and systematize our findings with the following contributions:

- **DeFi Reference Frame:** We provide the first framework for reasoning about DeFi system and threat models. We outline a wide spectrum of adversarial goals, assumptions, prior knowledge, capabilities, as well as common causes for potentially harmful DeFi incidents to create a standard model for related works (cf. Section II-A and II-B).
- **Gap Between Attackers and Defenders:** We analyze 181 DeFi incidents on Ethereum and BNB Smart Chain over a time frame of four years and structure the incidents, related academic papers, and security audit reports into a comprehensive taxonomy. We discover that academia and industry practices are underdeveloped with respect to the incident cause “unsafe DeFi protocol dependencies”, when compared to the practices of in-the-wild adversaries.
- **Incident Defense:** We investigate possible defense mechanisms against DeFi incidents. We show that SoTA similarity analysis can detect vulnerable and adversarial contracts. For instance, we identify 31/23 exactly matching vulnerable/adversarial contracts (i.e., bytecode similarity score of 100%)

when compared to previously known incidents. We also discover that 103 (56%) of the attacks are not executed atomically, granting a rescue time frame for defenders.

- **Tracing Source of Funds:** By tracking pre-incident adversarial footprints, we discover that 12(7.3%) and 21(8.0%) of the adversaries directly withdraw funds from exchange wallets, on Ethereum and BNB Smart Chain, respectively. Similarly, 55(21%) and 12(4.6%) of the attack funds stem directly from the US-sanctioned Tornado Cash mixer.

II. DeFi REFERENCE FRAME

Bitcoin is the first widely adopted permissionless system to allow users to send and receive financial value without the use of a third-party intermediary [19]. While Bitcoin also introduced the concept of smart contracts, more developer-friendly smart contract primitives [20] empowered the wide adoption of DeFi. DeFi currently provides a wide range of financial services such as exchanges [21], lending/borrowing [22]–[25], stablecoins [26], pegged tokens, price oracles [27], [28], mixing services [29], flash loans [8], yield farming [30], portfolio management, insurance [31], governance [32] etc. *Flash loans* allow traders to instantaneously request access to cryptocurrencies worth billions of USD. This is achieved through the creative use of the blockchain’s transaction atomicity property, through which a loan is not granted if the loan is not paid back with the due interests. Such convenient and programmable access to substantial capital has lowered the barrier of entrance for practical DeFi traders, as well as broaden the threat surface [8]. Because permissionless blockchains such as Bitcoin and Ethereum are known to not offer anonymity, but rather pseudonymity, alternative privacy-preserving blockchains emerged. These alternative blockchains break the linkability of addresses, by shuffling assets through an anonymity set. Notable solutions include ZCash which is based on zero-knowledge proofs [33], and Monero which is based on ring signatures and confidential transactions [34], [35]. Additionally, mixers operating as applications on existing blockchains emerged, such as Tornado Cash [29], Typhoon Network and AMR [36]. An extended background on DeFi, as well as a comparison to centralized finance (CeFi), is provided by related work [37]. In the following, we present a five-layer system model which is applicable to all DeFi incidents, as well as a threat model taxonomy based on various adversarial utilities, goals, knowledge, and capabilities.

A. System Model

As Figure 2 shows, our system model consists of five layers. The network layer enables data transmission between and among system layers. The blockchain consensus and smart contract layers enable financial services such as cryptocurrency trades to be performed without the use of trusted intermediaries. The protocol layer is a collection of DeFi protocols that are deployed and built on the smart contract layer. Note that on a permissionless blockchain, any DeFi user can create or deploy financial service protocols. Furthermore, DeFi protocols may rely on auxiliary services to increase the

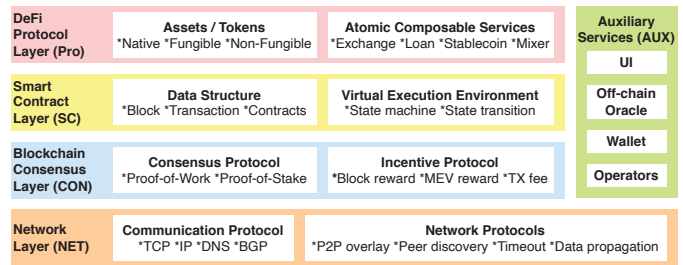


Fig. 2: High-level systematization of Decentralized Finance. DeFi is built on smart contract enabled blockchains, where auxiliary services help to ensure the overall efficiency, stability, and usability of the ecosystem. The network layer enables data transmission between and among system layers.

entire financial ecosystem’s efficiency, stability, and usability. We proceed to introduce the key components in each layer:

(i) Network Layer (NET):

- **Network Communication Infrastructure:** A communication protocol is a set of rules that allows two or more nodes in a system to communicate over a physical medium [38]. Users must rely on communication protocols such as TCP/IP, DNS, and BGP to interact with DeFi, whether directly through their own blockchain nodes or indirectly through third-party auxiliary services.
- **Blockchain and Peer-to-Peer (P2P) Network:** Blockchain network protocols instruct nodes on how to join, exit, and discover other nodes in the P2P network. A blockchain node may become unresponsive at any point in time, and related works observed frequent node churn [39]. Blockchain networks typically instruct each node to connect with many peers while also configuring a timeout to disconnect from non-responsive peers to ensure the network’s connectivity.
- **Front-running as a Service (FaaS):** Independent of the public blockchain P2P network, emerging centralized transaction propagation services offer an alternative option for traders to communicate to miners (e.g., Flashbots¹, Eden network², Bloxroute³, and Ethermine⁴). FaaS services allow DeFi traders to submit a bundle that consists of one or more transactions directly to FaaS miners without a broadcast on the P2P network. FaaS services may in addition provide bundle-level atomic state transition⁵, where the entire bundle is either executed successfully in the exact order that the transactions are provided, or fails collectively. Furthermore, FaaS traders are required to place a single sealed bid for the priority inclusion of the entire bundle, without observing the bid from other DeFi traders (i.e., sealed-bid auction). FaaS miners prioritize transaction bundles with the highest average bid at the top of the next mined block.

¹Flashbots: <https://blocks.flashbots.net/>

²Eden network: <https://www.edenetwork.io/>

³Bloxroute: <https://bloxroute.com/>

⁴Ethermine private RPC: <https://ethermine.org/private-rpc>

⁵This is different from transaction-level state transition in SC layer

(ii) Consensus Layer (CON):

- **Consensus Mechanism:** A consensus mechanism is a fault-tolerant mechanism in blockchain systems, which assist blockchain nodes to achieve the required agreement on a single data value or network state. The blockchain consensus mechanism typically consists of the following components:
 - A **Sybil attack-resistant leader election protocol**, such as Proof-of-Work (PoW) for Ethereum or Proof-of-Stake (PoS) for BNB Smart Chain;
 - A **consensus protocol** to synchronize the latest chain state (e.g., the longest chain with most difficulty); and
 - A **CON incentive mechanism**, which aims to encourage benign consensus activity. The *block reward* for instance, compensates every successful block appended to the main chain. *Transaction fees* are paid by transaction issuers to sequencers for inclusion in specific blocks and positions, and, lastly *blockchain extractable value (BEV)* and *miner extractable value (MEV)*, is potential extractable revenue left untouched [9], [40]–[44]. Transaction fees are typically enforced to be paid in the native blockchain coin.
- **Nodes and Their Operation Protocol:** A blockchain node may be responsible for one or several tasks: (i) transaction sequencing, specifying the order of transactions within a block; (ii) block generation; (iii) data verification; and (iv) data propagation. The two common types are:
 - **Sequencer nodes**, also known as miners in PoW blockchains, or validators in PoS blockchains, capture all four of the above responsibilities. Sequencers can insert, omit and reorder transactions in blocks they generate within the scope allowed by the protocol;
 - **Ordinary nodes** only perform blockchain data propagation and may perform data verification.

(iii) Smart Contract Layer (SC):

Despite the existence of different data storage structures (e.g., directed acyclic graph [45], sharding [46]–[49], etc.), SoTA smart contract enabled blockchains order their transactions as a linear sequence in order to achieve deterministic state transition [50]. In the following, we denote non-generic SC components with the asterisk mark (*). The remaining SC components are applicable to any DeFi systems.

- **Transactions:** A user specifies financial operations within a transaction to request blockchain state transitions. SC layer typically supports transaction-level *atomic state transition*, where all financial operations within the same transaction either execute in their entirety, or fail collectively.
- **State:** DeFi system state S specifies: (i) the cryptocurrency asset balances of users, (ii) the blockchain information, such as timestamps, coinbase addresses, block numbers, block gas limits (maximum computation unit per block), as well as (iii) the DeFi application state.
- **State Transition:** $\mathcal{T}(s \in S, tx \in TX) \rightarrow S$ is the state transition function returning a new state after executing tx , where TX denotes the set of all valid DeFi transactions.
- **Smart Contract:** A smart contract is code that is translated

into one or several state transition functions, which can then be triggered by a transaction. Smart contracts can also trigger the functions of other contracts. Upon deployment, a constructor function may initialize the contracts' state.

- **Block State Transition*:** Both Ethereum and BNB smart chain record transactions with an ordered list of blocks. We denote B as the set of blocks, and use $b_i \in B$ to denote a block at height i . Each block b_i may include a list of n transactions, denoted by $\{tx_{b_i}^0, \dots, tx_{b_i}^n\}$, $n \geq 0$. A block state $\mathcal{S}(b_{i+1})$ stems from the sequential execution of all transactions in block b_{i+1} on $\mathcal{S}(b_i)$ (cf. Equation 1).

$$\mathcal{S}(b_{i+1}) = \mathcal{T}(\dots \mathcal{T}(\mathcal{T}(\mathcal{S}(b_i), tx_{b_{i+1}}^0), tx_{b_{i+1}}^1) \dots) \quad (1)$$

- **SC and Layer 2 Blockchain (L2) Incentive Mechanism*:** DeFi protocols can operate on so-called L2 systems, such as side-chains⁶, commit-chains [51] or its inspired successor optimistic-rollups [52], and zk-rollups⁷. Because L2 systems are created on top of Layer 1 blockchains (also known as L1, e.g. Ethereum and BNB Smart Chain), L2 systems often implement their consensus incentive mechanisms on L1 blockchains' SC layer to encourage benign activities [53].

(iv) DeFi Protocol Design Layer (PRO):

- **Cryptocurrency Protocols:** DeFi supports a variety of asset standards, which define a common set of rules and interfaces for the transfer and approval of cryptocurrency assets (e.g., ERC-20 [54]). DeFi protocols may, however, deviate from the common standard by proposing a newer variant with domain-specific features. The Ampleforth protocol is an example of a custom asset standard, which dynamically adjusts its total token supply to maintain a stable price (i.e., stablecoins) [55]. Newer standards may remain backward compatible, while extending the feature set (e.g., ERC-777 enables the injection of state transitions, i.e., hooks, during transfer calls [56]). Note that backward-compatible standards may however violate the security assumptions of existing protocols, thus empowering novel attack vectors.
- **Financial Protocols:** While DeFi protocols may appear inspired by traditional financial services, the blockchains' unique features (e.g., transparency, atomicity, and discrete batch transaction execution) enable novel designs. For instance, unlike CeFi, DeFi platforms are notably intertwined through atomic composability. For instance, leveraged liquidity mining protocols such as Alpha Homora [57] and Harvest Finance [58] integrate automated market makers (i.e., Uniswap [2]) and lending platforms (i.e., Compound [6]).
- **Protocol Layer Incentive Mechanism:** DeFi protocols may introduce PRO incentive mechanisms to encourage desired user behavior. One example is the airdrop of governance tokens in exchange for providing liquidity in decentralized exchanges [59], [60] (e.g., Sushiswap⁸ and Curve⁹).

⁶For example, Polygon network (<https://polygon.technology/>)

⁷For example, zkSync (<https://zksync.io/>)

⁸Sushiswap staking: <https://app.sushi.com/stake>

⁹Curve staking: <https://resources.curve.fi/crv-token/staking-your-crv>

Capability Description	Knowledge
C_{NET}^1 $\mathbb{A}$ may control network service providers (e.g., DNS).	K_3
C_{NET}^2 $\mathbb{A}$ may manipulate incoming messages to deceive a node's perception of current state (e.g., eclipse attacks [61]).	K_1 or K_3
C_{NET}^3 $\mathbb{A}$ may censor or delay the transmission of messages. For example in selfish mining, $\mathbb{A}$ may not broadcast the blocks appended to the competing chain [62].	K_1 or K_3
C_{NET}^4 $\mathbb{A}$ may transmit transactions to miners using FaaS.	K_1
C_{CON}^1 $\mathbb{A}$ may fork or append on a forked chain in an attempt to catch up and overwrite the longest chain.	K_2
C_{CON}^2 $\mathbb{A}$ may censor mempool transaction temporarily.	K_2
C_{CON}^3 $\mathbb{A}$ may (i) include, exclude, or re-order transactions within its blocks if $\mathbb{A}$ is/colludes with a sequencer, or (ii) engage in front/back-running [40], [41], [43].	K_1 or K_2
C_{SC}^1 $\mathbb{A}$ may simulate state transition off-chain (cf. Equation 1) with any arbitrary transactions on forked blockchain states, instead of issuing transactions on-chain.	K_1
C_{PRO}^1 $\mathbb{A}$ may use mixer services to break account linkability.	K_1
C_{PRO}^2 $\mathbb{A}$ may borrow, use, and return liquidity from a decentralized cryptocurrency pool within a single atomic transaction using a flash loan [8].	K_1
C_{PRO}^3 $\mathbb{A}$ may compose the state transition from multiple DeFi protocols (composability).	K_1
C_{PRO}^4 $\mathbb{A}$ may compose all state transitions required in one single transaction, and execute atomically.	K_1
C_{PRO}^5 $\mathbb{A}$ may deploy or utilise a customised contract, which mimics the function interface (i.e., abi) of one or many DeFi protocols.	K_1
C_{3RD}^1 $\mathbb{A}$ may manipulate external oracle data [14].	K_3
C_{3RD}^2 $\mathbb{A}$ may compromise the wallet passphrase of specific DeFi users, operators and etc.	K_3

TABLE I: Adversarial capabilities and knowledge level at each layer of our system model.

Raw on-chain data	Raw P2P network data	Public side channel	Public data analysis	Private mempool	Sequencing rules	Next block early access	Oracle update early access	External price early access	Wallet passphrase access	Other miscellaneous	Knowledge
✓	✓	✓	✓	×	△	×	×	×	×	×	Public (K_1)
✓	✓	✓	✓	✓	✓	×	×	×	×	×	Sequencer (K_2)
✓	✓	✓	✓	×	×	×	△	△	△	△	Insider (K_3)

TABLE II: Categorization of adversarial knowledge levels. “✓” has access, “×

(v) Auxiliary Service Layer (AUX):

Auxiliary services refer to any entity that is required or which facilitates DeFi's efficiency, but does not belong to any of the four above-mentioned system layers (i.e., NET, CON, SC, and PRO). For example, an operationally active DeFi protocol implementation may consist of: (i) front-end code; (ii) project developers realizing the protocol designs; (iii) “operators” with administrative powers, such as the privilege to deploy the code, upgrade the protocol, freeze or cease the activity of the operative DeFi protocol; (iv) off-chain oracle services which sync price data from centralized exchanges to on-chain smart contracts, etc.

B. Threat Model Taxonomy

In the following we provide a holistic view of the adversarial utilities, goals, knowledge and capabilities, to engender a common reference frame which we subsequently apply in Section III to relatively compare all observed DeFi attacks.

(i) What is a DeFi Incident: An incident refers to a series of actions that result in an unexpected financial loss to one or more of the following entities: (i) users; (ii) liquidity providers; (iii) speculators; or (iv) operators. We classify incidents into the following two types:

- **Attacks:** An adversary, $\mathbb{A}$, may exploit vulnerabilities, in an attempt to disable, delay, or alter a DeFi protocol's expected state transition. Despite the fact that vulnerabilities exist on all five system layers, DeFi vulnerabilities are most commonly found in the following three layers (cf. Table III):

- 1) *SC Layer Vulnerabilities* result from coding mistakes, such as arithmetic error, casting error, inconsistent access control, function reentrancy, etc;
- 2) *PRO Layer Vulnerabilities* may resemble financial market manipulation instead of traditional system vulnerabilities (i.e., protocol design flaws, such as unsafe external protocol dependency or interactions). Yet, the practitioners' community as well as related works [8] classify market manipulations as attacks, which necessarily require a vulnerable system or system state; and
- 3) *AUX Layer Vulnerabilities*, which includes both operational vulnerability (e.g., off-chain oracle manipulation, compromised private key, etc.) and “information asymmetry” attacks (e.g., backdoor, honeypot, phishing, etc.). Generally speaking, we observe that users may not always (or may not be able to) inspect and understand a DeFi protocol smart contract before providing financial assets, let alone evaluating its security and risks [63], [64]. As such, a user's understanding of a contract operation may be mostly based on marketing communications, rather than the factual contract source code, leading the user to unforeseen or unexpected circumstances.

- **Accidents:** Any incident that does not explicitly involve proactive adversaries is classified as a DeFi accident. For example, a user's fund may become permanently locked in a contract due to unintentional coding mistakes.

(ii) Adversarial Utility and Goal: Throughout this work, we assume that $\mathbb{A}$ is a rational agent aiming to maximize its utility. We categorize utility into the following two categories:

- **U_1 -Monetary:** The most common utility we find is of monetary nature. We define the monetary utility function as the total increase in market value of $\mathbb{A}$'s cryptocurrency asset portfolio, which $\mathbb{A}$ aims to maximize.
- **U_2 -Non-monetary:** $\mathbb{A}$ may instead maximize non-monetary utilities, such as sense of accomplishment, or reputation. DeFi white hat hackers (also known as ethical hackers) are an example of a non-monetary adversary, as they attack in an attempt to minimize the loss from DeFi incidents.

(iii) **Adversarial Knowledge:** Table II differentiates between the following three types of adversarial knowledge.

- K_1 -**Public:** $\mathbb{A}$ can access public information, including: (i) Raw on-chain data such as blocks, uncle blocks, transactions, accounts, balances, and deployed smart contract bytecode; (ii) Raw network data, such as P2P network transactions, pending blocks, discarded stale blocks, blockchain node IP addresses, port numbers, client version strings, etc; (iii) Public side channel, such as, open-source smart contract code, social media/chat messages; (iv) Public data analysis, such as inferred network topology, estimated sequencer location, and decompiled smart contract bytecode [65].
- K_2 -**Sequencer:** $\mathbb{A}$ obtains the following information, if $\mathbb{A}$ is/colludes with a sequencer: (i) Pending transactions from private communication channels; (ii) Transaction ordering logic for the corresponding sequencer, including bribery preferences; (iii) Early access to block state before broadcast if the corresponding sequencer generates the next block.
- K_3 -**Insider:** Privileged information asymmetry may occur for example if $\mathbb{A}$ has early access to external market prices, oracle updates, or the wallet passphrases of an operator¹⁰.

(iv) **Adversarial Capabilities:** Table I outlines the adversarial capabilities and required knowledge. Note that $\mathbb{A}$ with differing levels of knowledge may be able to achieve the same capability. Sequencers, for example, can control the transaction order of their generated blocks (K_2), whereas $\mathbb{A}$ without sequencer knowledge can also perform front-/back-running by competing on the public blockchain P2P network (K_1).

III. DATA

In this section we present our methodology to sample a dataset of “works under investigation”, including research papers, security tools (i.e., intrusion detection, intrusion prevention and vulnerability detection), audit reports, and real-world incidents. We manually label which incident types each work addresses (cf. Table III and IX). Our dataset serves as the foundation for the analysis in Sections IV, V and VI.

Academic Papers: We identify relevant papers in eight of the top security, software engineering, and programming language conferences (i.e., SSP, CCS, NDSS, USENIX, ICSE, ASE, POPL, PLDI) from 2018 to 2021. Our methodology first crawls papers using Google Scholar’s keyword search¹¹, and then performs backward and forward reference searches to find additional relevant works. Our dataset omits: (i) papers irrelevant to DeFi, such as Bitcoin specific attacks or Monero privacy; and (ii) DeFi related papers that do not address any particular type of incidents, such as contract patching [66], model checking [67], bug bounties [68], and reverse engineering [65]. In total, our dataset captures 7 relevant surveys and

SoKs, 29 security tools, and 42 attack papers. We manually label the incident types addressed in each academic paper and cross-validate our labels against the related works section.

Audit Reports: We collect and manually inspect 30 recent public audit reports from 6 security testing companies (Beosin, PeckShield, Slowmist, Consensys, Certik, Trial of Bits). We notice that the reports collected perform manual auditing and may not explicitly disclose what the auditors examined. For example, while each of the six companies checked the common vulnerability “inconsistent access control” in at least one audit report, only 19 of the 30 (63%) audit reports explicitly state it. For reproducibility and objectiveness, we can only be certain that an audit has addressed an incident type, if it: (i) explicitly warns about the risk of a potential incident, or (ii) explicitly states that the code passed the check of an incident type. This methodology, however, leads to an underestimation of the absolute number of incident types addressed in the audit reports¹². Note that we are only attempting to quantify whether practitioners address certain incident types less frequently than the others, and therefore this unbiased underestimation should have no significant impact on our analysis.

Incidents: Our dataset consists of 117 and 69 incidents on Ethereum and BSC respectively (in total 181 incidents) over a period of four years from Apr 30, 2018 to Apr 30, 2022. These incidents are gathered from the following three sources¹³: (i) Rekt News; (ii) Slowmist; and (iii) Cryptosec. We exclude non-DeFi incidents, such as blockchain-based gambling and gaming applications. The incidents of which we cannot identify the adversary are also excluded. We construct the following features for each of the incident:

- **Incident Type and Cause:** We manually label the type and cause of each incident (cf. Table III for incidents taxonomy, which is further discussed in Section VI). It should be noted that we may associate one incident with multiple types or causes across multiple system layers.
- **Adversaries:** When we can identify an incident’s adversaries, we manually classify adversarial goal, knowledge, and capability based on our reference frame (cf. Section II).
- **Averaged Total Monetary Loss (in USD):** The most perceptible impact of harm is direct monetary loss. We collect the total monetary loss reported by the aforementioned data sources, where the victim can be either users, liquidity providers, speculators, or protocol operators. When applicable, we cross-validated the loss with on-chain transaction data, and then remove sources that report incorrect loss¹⁴.
- **Cumulative Abnormal Return (CAR):** CAR reflects harm by measuring how token price responds to an incident. We

¹²As an example, Trial of Bits does check for PRO layer incidents in other audit reports, such as sandwich in TOB-Computable-018 (<https://github.com/trailofbits/publications/blob/master/reviews/computable.pdf>), replay in TOB-HERMEZ-014 (<https://github.com/trailofbits/publications/blob/master/reviews/hermez.pdf>), etc., but are not included in our sampled dataset.

¹³Correspondingly: (i) <https://rekt.news/>; (ii) <https://hacked.slowmist.io/en/>; and (iii) <https://cryptosec.info/defi-hacks/>

¹⁴We rely on Uniswap, Sushiswap, Pancakeswap and Bakery swap as our price oracles when validating on-chain transaction

TABLE III: DeFi incidents taxonomy. We label the incident types that each academic paper and auditing report address. We also group the incidents that occur in the wild. Despite that this table focuses on Ethereum and BSC, we anticipate the taxonomy remains generic and thus applicable to all DeFi enabled blockchains. ● - Incident type addressed; ■ - Incident type checked (likely with tools); ○ - Incident type checked (manually). Note that we can only be sure that an incident type has been addressed if an auditing report: (i) explicitly warns of the risk of a potential incident, or (ii) explicitly states that the code passed the check of an incident type. We visualize the gaps using a heat map, where a darker colour indicates a greater frequency of occurrences.

Incident Cause	Incident Type	SoKs, Surveys	Tools	Academic Papers (We abbreviate Userx Security as UNX)	Papers	Boson	PeckShield	SlowMist	Consensus	Certik	Trail of Bits	Gap Visualization
Network	Network layer transparency	10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 1011, 1012, 1013, 1014, 1015, 1016, 1017, 1018, 1019, 1020, 1021, 1022, 1023, 1024, 1025, 1026, 1027, 1028, 1029, 1030, 1031, 1032, 1033, 1034, 1035, 1036, 1037, 1038, 1039, 1040, 1041, 1042, 1043, 1044, 1045, 1046, 1047, 1048, 1049, 1050, 1051, 1052, 1053, 1054, 1055, 1056, 1057, 1058, 1059, 1060, 1061, 1062, 1063, 1064, 1065, 1066, 1067, 1068, 1069, 1070, 1071, 1072, 1073, 1074, 1075, 1076, 1077, 1078, 1079, 1080, 1081, 1082, 1083, 1084, 1085, 1086, 1087, 1088, 1089, 1090, 1091, 1092, 1093, 1094, 1095, 1096, 1097, 1098, 1099, 1100, 1101, 1102, 1103, 1104, 1105, 1106, 1107, 1108, 1109, 1110, 1111, 1112, 1113, 1114, 1115, 1116, 1117, 1118, 1119, 1120, 1121, 1122, 1123, 1124, 1125, 1126, 1127, 1128, 1129, 1130, 1131, 1132, 1133, 1134, 1135, 1136, 1137, 1138, 1139, 1140, 1141, 1142, 1143, 1144, 1145, 1146, 1147, 1148, 1149, 1150, 1151, 1152, 1153, 1154, 1155, 1156, 1157, 1158, 1159, 1160, 1161, 1162, 1163, 1164, 1165, 1166, 1167, 1168, 1169, 1170, 1171, 1172, 1173, 1174, 1175, 1176, 1177, 1178, 1179, 1180, 1181, 1182, 1183, 1184, 1185, 1186, 1187, 1188, 1189, 1190, 1191, 1192, 1193, 1194, 1195, 1196, 1197, 1198, 1199, 1200, 1201, 1202, 1203, 1204, 1205, 1206, 1207, 1208, 1209, 1210, 1211, 1212, 1213, 1214, 1215, 1216, 1217, 1218, 1219, 1220, 1221, 1222, 1223, 1224, 1225, 1226, 1227, 1228, 1229, 1230, 1231, 1232, 1233, 1234, 1235, 1236, 1237, 1238, 1239, 1240, 1241, 1242, 1243, 1244, 1245, 1246, 1247, 1248, 1249, 1250, 1251, 1252, 1253, 1254, 1255, 1256, 1257, 1258, 1259, 1260, 1261, 1262, 1263, 1264, 1265, 1266, 1267, 1268, 1269, 1270, 1271, 1272, 1273, 1274, 1275, 1276, 1277, 1278, 1279, 1280, 1281, 1282, 1283, 1284, 1285, 1286, 1287, 1288, 1289, 1290, 1291, 1292, 1293, 1294, 1295, 1296, 1297, 1298, 1299, 1300, 1301, 1302, 1303, 1304, 1305, 1306, 1307, 1308, 1309, 1310, 1311, 1312, 1313, 1314, 1315, 1316, 1317, 1318, 1319, 1320, 1321, 1322, 1323, 1324, 1325, 1326, 1327, 1328, 1329, 1330, 1331, 1332, 1333, 1334, 1335, 1336, 1337, 1338, 1339, 1340, 1341, 1342, 1343, 1344, 1345, 1346, 1347, 1348, 1349, 1350, 1351, 1352, 1353, 1354, 1355, 1356, 1357, 1358, 1359, 1360, 1361, 1362, 1363, 1364, 1365, 1366, 1367, 1368, 1369, 1370, 1371, 1372, 1373, 1374, 1375, 1376, 1377, 1378, 1379, 1380, 1381, 1382, 1383, 1384, 1385, 1386, 1387, 1388, 1389, 1390, 1391, 1392, 1393, 1394, 1395, 1396, 1397, 1398, 1399, 1400, 1401, 1402, 1403, 1404, 1405, 1406, 1407, 1408, 1409, 1410, 1411, 1412, 1413, 1414, 1415, 1416, 1417, 1418, 1419, 1420, 1421, 1422, 1423, 1424, 1425, 1426, 1427, 1428, 1429, 1430, 1431, 1432, 1433, 1434, 1435, 1436, 1437, 1438, 1439, 1440, 1441, 1442, 1443, 1444, 1445, 1446, 1447, 1448, 1449, 1450, 1451, 1452, 1453, 1454, 1455, 1456, 1457, 1458, 1459, 1460, 1461, 1462, 1463, 1464, 1465, 1466, 1467, 1468, 1469, 1470, 1471, 1472, 1473, 1474, 1475, 1476, 1477, 1478, 1479, 1480, 1481, 1482, 1483, 1484, 1485, 1486, 1487, 1488, 1489, 1490, 1491, 1492, 1493, 1494, 1495, 1496, 1497, 1498, 1499, 1500, 1501, 1502, 1503, 1504, 1505, 1506, 1507, 1508, 1509, 1510, 1511, 1512, 1513, 1514, 1515, 1516, 1517, 1518, 1519, 1520, 1521, 1522, 1523, 1524, 1525, 1526, 1527, 1528, 1529, 1530, 1531, 1532, 1533, 1534, 1535, 1536, 1537, 1538, 1539, 1540, 1541, 1542, 1543, 1544, 1545, 1546, 1547, 1548, 1549, 1550, 1551, 1552, 1553, 1554, 1555, 1556, 1557, 1558, 1559, 1560, 1561, 1562, 1563, 1564, 1565, 1566, 1567, 1568, 1569, 1570, 1571, 1572, 1573, 1574, 1575, 1576, 1577, 1578, 1579, 1580, 1581, 1582, 1583, 1584, 1585, 1586, 1587, 1588, 1589, 1590, 1591, 1592, 1593, 1594, 1595, 1596, 1597, 1598, 1599, 1600, 1601, 1602, 1603, 1604, 1605, 1606, 1607, 1608, 1609, 1610, 1611, 1612, 1613, 1614, 1615, 1616, 1617, 1618, 1619, 1620, 1621, 1622, 1623, 1624, 1625, 1626, 1627, 1628, 1629, 1630, 1631, 1632, 1633, 1634, 1635, 1636, 1637, 1638, 1639, 1640, 1641, 1642, 1643, 1644, 1645, 1646, 1647, 1648, 1649, 1650, 1651, 1652, 1653, 1654, 1655, 1656, 1657, 1658, 1659, 1660, 1661, 1662, 1663, 1664, 1665, 1666, 1667, 1668, 1669, 1670, 1671, 1672, 1673, 1674, 1675, 1676, 1677, 1678, 1679, 1680, 1681, 1682, 1683, 1684, 1685, 1686, 1687, 1688, 1689, 1690, 1691, 1692, 1693, 1694, 1695, 1696, 1697, 1698, 1699, 1700, 1701, 1702, 1703, 1704, 1705, 1706, 1707, 1708, 1709, 1710, 1711, 1712, 1713, 1714, 1715, 1716, 1717, 1718, 1719, 1720, 1721, 1722, 1723, 1724, 1725, 1726, 1727, 1728, 1729, 1730, 1731, 1732, 1733, 1734, 1735, 1736, 1737, 1738, 1739, 1740, 1741, 1742, 1743, 1744, 1745, 1746, 1747, 1748, 1749, 1750, 1751, 1752, 1753, 1754, 1755, 1756, 1757, 1758, 1759, 1760, 1761, 1762, 1763, 1764, 1765, 1766, 1767, 1768, 1769, 1770, 1771, 1772, 1773, 1774, 1775, 1776, 1777, 1778, 1779, 1780, 1781, 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793, 1794, 1795, 1796, 1797, 1798, 1799, 1800, 1801, 1802, 1803, 1804, 1805, 1806, 1807, 1808, 1809, 1810, 1811, 1812, 1813, 1814, 1815, 1816, 1817, 1818, 1819, 1820, 1821, 1822, 1823, 1824, 1825, 1826, 1827, 1828, 1829, 1830, 1831, 1832, 1833, 1834, 1835, 1836, 1837, 1838, 1839, 1840, 1841, 1842, 1843, 1844, 1845, 1846, 1847, 1848, 1849, 1850, 1851, 1852, 1853, 1854, 1855, 1856, 1857, 1858, 1859, 1860, 1861, 1862, 1863, 1864, 1865, 1866, 1867, 1868, 1869, 1870, 1871, 1872, 1873, 1874, 1875, 1876, 1877, 1878, 1879, 1880, 1881, 1882, 1883, 1884, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1892, 1893, 1894, 1895, 1896, 1897, 1898, 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1909, 1910, 1911, 1912, 1913, 1914, 1915, 1916, 1917, 1918, 1919, 1920, 1921, 1922, 1923, 1924, 1925, 1926, 1927, 1928, 1929, 1930, 1931, 1932, 1933, 1934, 1935, 1936, 1937, 1938, 1939, 1940, 1941, 1942, 1943, 1944, 1945, 1946, 1947, 1948, 1949, 1950, 1951, 1952, 1953, 1954, 1955, 1956, 1957, 1958, 1959, 1960, 1961, 1962, 1963, 1964, 1965, 1966, 1967, 1968, 1969, 1970, 1971, 1972, 1973, 1974, 1975, 1976, 1977, 1978, 1979, 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 1989, 1990, 1991, 1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 219										

expect rational investors' risk aversion to information shocks will diverge the token price in the equilibrium and lead to abnormal returns (ARs) [69], [70]. We choose the capital asset pricing model (CAPM) as the benchmark for normal returns. We refer interested readers to Section B in the appendix for the detailed steps of deriving CAR.

- **Total Value Locked (in USD):** TVL is calculated as the product of the total token balance held by a protocol's smart contracts and token price in USD [168]. Greater TVL indicates greater value of assets that can be potentially compromised under DeFi incidents. We attain the pre-attack TVL for 126 incidents using DeBank¹⁵ and DeFiLlama [1].
- **Audit Status:** For each incident, we manually search auditing histories from the following four sources: (i) a protocol's website; (ii) a protocol's social media and blog post (e.g., Twitter and Medium); (iii) public git repositories; (iv) a search engine (i.e., Google). We then label each incident according to the following rules: (*Audited*): the victim smart contract is audited prior to the incident; (*Partially Audited*): audits are performed before the incident, but not for the specific victim smart contract or for an older version; and (*Not Audited*): no audit history is found prior to the incident.
- **Emergency Pause, Disclosure and Reimbursement:** We crawl the following three features in an attempt to measure a protocol's reactive defense: (a) Did the protocol disclose the incident within 20 days?¹⁶ (b) Has the protocol reimbursed its users within 20 days? and (c) Did the protocol execute a circuit breaker [169] or emergency pause? We manually search for auditing histories from the following three sources: (i) public announcements on a protocol's website; (ii) a protocol's social media and blog post (e.g., Twitter and Medium); and (iii) the protocol's main discussion forum.

Limitations: Our methodology has the following limitations:

- **Soundness:** Because our data crawling process is heavily reliant on manual labor, human errors may occur. To mitigate this limitation, we cross-validate our data with external sources whenever possible. Additionally, we conduct internal data reviews through pull requests. Each incident is reviewed by at least two paper authors before the pull request is merged.
- **Completeness:** - Despite that Ethereum and BSC account for 77% of the total DeFi NVL (cf. Section I), incidents' features, such as adversarial behavior and deployed defense, on other DeFi enabled blockchains can differ. To ensure the paper's reproducibility, we only consider fully disclosed incidents that can be found through public sources. While incomplete, this DeFi incident dataset is the largest available collection that we are aware of.
- **Bias:** - Our incidents dataset is gathered from three publicly sources (e.g., Rekt News, Slowmist and Peckshield). These three sources are, to our knowledge, the most extensive DeFi incident databases accessible. Unfortunately, none of these three sources explicitly document their data collection

¹⁵<https://open.debank.com/>, accessed on September 30, 2021

¹⁶We choose a custom time frame as an example.

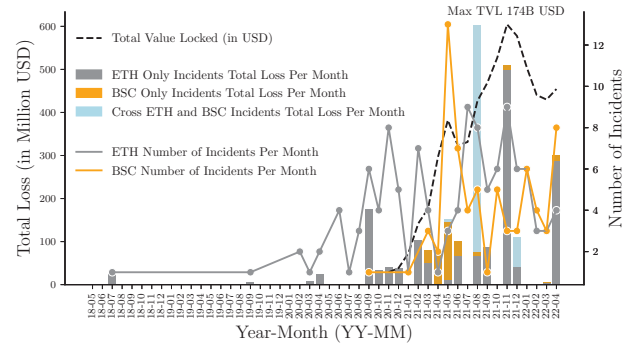


Fig. 3: Monthly number of DeFi incidents and total loss (in million USD) for Ethereum and BNB Smart Chain from Apr 30, 2018 to Apr 30, 2022, in comparison to the total value locked. According to our data, the frequency, and monthly loss increase as the TVL increases.

process. As a result, we are unable to evaluate whether these sources contain bias, and our dataset may therefore inherit the sampling bias from these sources¹⁷.

IV. ANALYSIS

A. Incident Frequency

Figure 3 shows the monthly number of incidents in relation to the total monthly loss. We find that the majority of the DeFi incidents occur after late 2020, with the peak in August 2021, when nearly 600 million dollars are lost in a single month.

Despite the fact that BSC is a relatively new blockchain, it experienced 69 DeFi incidents. We discover that 29 of the BSC incidents are exploiting PRO layer design flaws. In particular, between the 19th of May and the 3rd of June 2021, we observe recurring exploits on a group of forked protocols¹⁸. The time frame of 15 days suggests that attackers do not yet have automated tools to scan and reproduce similar incidents.

Figure 4 illustrates the incident frequency per group and the involved system layer. Overall, we find that the frequency of all incident types increase over time from 3.1 per month in 2020 to 8.5 per month in the first four months of 2022 on average (2.74 $\times$). We also observe that the most common incident cause are SC Layer (42%), PRO Layer (40%), and AUX Layer (30%) vulnerabilities.

B. DeFi Protocol Types

Table IV groups the incidents that we collect based on their protocol/application type. We find that yield farming protocols and cross-chain bridges incur 44% of the total

¹⁷For example, our methodology only includes BEV incidents disclosed in the three abovementioned sources. For detailed BEV studies, we refer the interested reader to the rich corpus of previous works [40]–[43], [101]

¹⁸PancakeBunny suffered a performance fee minting attack on May 19, 2021, where the adversary manipulated the on-chain oracle and siphoned \$45M in profit. Within two weeks, the copycats Autoshark, MerlinLab and PancakeHunny were exploited in a similar fashion: the adversary (i) exploited the vulnerability of mintFor/mintForV2 function to manipulate LP token prices and (ii) used cross-chain bridge and TC to launder money.

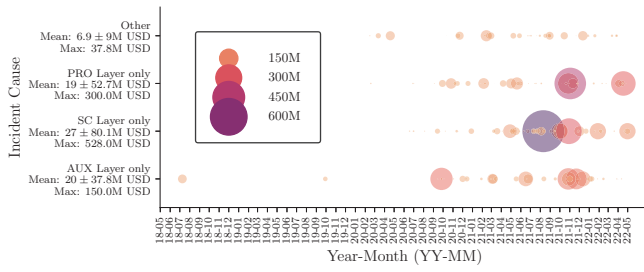


Fig. 4: Loss (in million USD) and frequency of DeFi incidents on Ethereum and BNB Smart Chain from Apr 30, 2018 to Apr 30, 2022 grouped by incident cause. Each circle represents a unique incident, and the size of the circle is proportional to the estimated monetary loss in USD.

	Yield	Bridge	Lending	DEX	Stablecoin	DAO	Payment	Derivatives	Insurance	Others
Is the monetary loss related to the type of the DeFi protocol?										
Loss (in M USD)	868	860	485	450	286	200	72	32	14	713
Pct. of Total Loss	22%	22%	13%	12%	7%	5%	2%	1%	0%	18%
Is the number of the security incidents related to the type of the DeFi protocol?										
Num. of Incidents	50	10	22	28	7	7	7	6	3	49
Pct. of Incidents	27%	5%	12%	15%	4%	4%	4%	3%	2%	27%
TVL (in B USD)	9.2	11.4	18.2	27.7	-	-	0.5	2.2	0.6	-
Is the vulnerability type related to the type of the DeFi protocol?										
SC layer related	48%	60%	50%	39%	43%	0%	0%	50%	33%	43%
AUX layer related	20%	30%	18%	29%	0%	43%	71%	50%	33%	47%
PRO layer related	52%	10%	59%	39%	86%	29%	43%	17%	33%	24%
NET layer related	0%	0%	5%	4%	0%	14%	0%	0%	0%	2%

TABLE IV: Loss (in million USD) and frequency of DeFi incidents grouped by application type, on Ethereum and BNB Smart Chain from Apr 30, 2018 to Apr 30, 2022. We crawl TVL for each category from DeFiLlama on Aug 6, 2022.

monetary loss, although their total TVL is only 20.6 billion USD (30.2%). In contrast, DEX protocols have the biggest TVL (27.7 billion USD, 40.6%), but have only incurred a loss of 450 million (12%). In addition, we observe that the distribution of vulnerabilities varies per protocol type. For example, 86% and 59% of the incidents related to stablecoins and lending involve PRO layer vulnerabilities respectively, which is significantly higher than other protocol types.

C. Structural Equation Modeling

In this section, we apply *Structural Equation Modeling* (SEM) [170]–[180] to test and measure causal relationships between variables (cf. Figure 5 and Table V).

- **What is SEM:** SEM refers to a collection of techniques to examine “latent variables” that are assumed to exist but cannot be directly observed. In more detail, SEM is a multivariate analysis technique that supports a flexible hybrid of confirmatory factor analysis (CFA) [181]–[183] and latent structural regression [180], [184]. An SEM model encompasses two sub-models [185] (cf. Equation 2): (i) a *measurement model* that conducts CFA to test the hypothesized relationships between a given latent variable and its corresponding observed variables; and (ii) a *structural model*

Latent Variable	Observed Variable	Description
Preventive Defense	PD1	Was the victim protocol audited before the incident?
	PD2	Does the victim protocol support emergency pause?
Asset	A1	Total value locked (TVL, in USD)
Reactive Defense	RD1	Duration between incident occurrence and emergency pause
	RD2	Was the incident disclosed?
Harm	H1	Cumulative abnormal return (CAR) (in %)
	H2	Total monetary loss (in USD)

TABLE V: Latent and observed variables we construct in structural equation modeling (SEM).

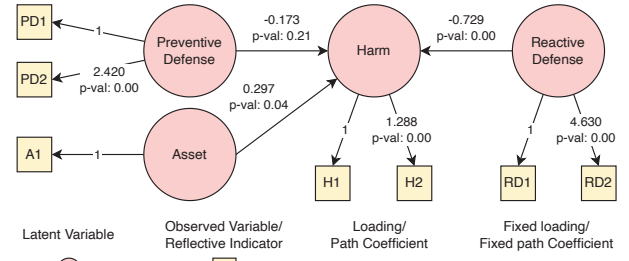


Fig. 5: Structural equation model (SEM) after fitting.

that performs latent structural regression to infer the causal relationships between different latent variables.

$$\begin{cases} \eta = B\eta + \varepsilon & (\text{structural model}) \\ y = \Lambda\eta + \delta & (\text{measurement model}) \end{cases} \quad \text{where:} \quad (2)$$

η and y are vectors of latent and observed variables; ε and δ are independent error terms.

- **Why SEM:** The literature [186] utilized SEM to study latent variables in cyber risks. In this work, we apply similar techniques to measure the causal relationships in DeFi incidents. To this end, we do not consider approaches that are unable to support causal inference in the presence of latent variables, such as linear mixed models [187] and dimensional reduction techniques [188]. Previous literature suggests the causal Bayesian network being the best alternative to SEM. However, it requires at least 1000 samples to get a satisfactory performance. With limited samples of DeFi incidents, we consider SEM a more suitable approach.
- **Specification:** Our model consists of four latent variables, including one endogenous/dependent variables (i.e., *harm*), and three exogenous/independent variables (i.e., *asset*, *preventive defense* and *reactive defense*). We measure one or two observed variables for each latent variable (cf. Table V). To construct the causal graph, we employ a variation of the hypothesis by Wood and Böhme [186]: *preventive defense*, *reactive defense* and *asset* jointly affect *harm*.
- **Estimation:** We utilize a logarithmic price scale to transform monetary values (e.g., TVL and monetary loss). We then further apply min-max normalization to convert continuous variables to values in range [0, 1]. Categorical values

Duration after the incident starts	$\leq 1h$	$\leq 6h$	$\leq 12h$	$\leq 24h$	$\leq 48h$
Number of protocols	1	24	11	7	8
Percentage (out of 87 protocols)	2.2%	47%	22%	14%	16%

TABLE VI: We quantify the speed at which DeFi protocols execute their emergency pause. Out of the 51 DeFi protocols that allow an emergency pause, the fastest has initiated a pause within the first hour of an incident.

are mapped into ordinal values^{19,20,21}. We fit our SEM using an open-sourced library, *semopy* [185] (cf. Figure 5).

- **Fitness:** Our model is examined using a collection of indices, including (i) the adjusted Chi-square ($\frac{\chi^2}{DoF}$) [189]; (ii) goodness of fit index (GFI) [172]; (iii) comparative fit index (CFI) [190]; and (iv) normed fit index (NFI) [191]. The majority of indices conform to their commonly accepted value in the literature except adjusted Chi-square²².
- **Analysis:** Our findings suggest that the latent variable “harm” increases with “asset exposure”, which conforms with previous security research. We also find that harm decreases if the latent variable “reactive defense” increases. To our surprise, the p-value for preventive defense is high (0.21), meaning that our model does not find strong evidence to suggest preventive defense reduces harm.
- **Limitations:** Our primary limitation is the relatively small sample size. In the event that the number of DeFi incidents increases in the future, our model should be re-evaluated and cross-validated using additional causal experiments.

D. Emergency Pause

DeFi protocols may support an emergency pause, which is analogous to circuit breakers [169] in conventional centralized exchanges. This section examines the speed at which DeFi protocols initiate an emergency pause (cf. Table VI). According to our data, 87 of the 183 victims support the emergency pause mechanism (47.5%). However, only 51 of the 87 protocols (58.6%) pause their protocol within 48 hours, and only one protocol pauses within the first hour of the incident. Our statistics suggest that DeFi protocols may not yet have just-in-time intrusion detection mechanisms to identify abnormal protocol states or malicious transactions, which limits the effectiveness of an emergency pause mechanism.

E. Effectiveness of Security Audits

Section IV-C studies the influence of security audits on harm, by performing causal inference analysis (e.g., SEM) on past incidents only. In the following section, we will attempt to estimate the effectiveness of security audits.

- **Additional Crawling** To measure security audit effectiveness, we: (i) Crawl all DeFi protocols via DeFiLama’s API [1]. Out of the 1080 protocols listed on DeFiLama, 776

¹⁹PD1: {Not Audited $\rightarrow$ 0, Partially Audited $\rightarrow$ 0.5, Audited $\rightarrow$ 1}

²⁰PD2: {No Emergency Pause $\rightarrow$ 0, Supports Emergency Pause $\rightarrow$ 1}

²¹RD2: {Not Disclosed $\rightarrow$ 0, Disclosed $\rightarrow$ 1}

²²Previous works suggest that the fit should be ≤ 5 for adjusted Chi-square [192], and ≥ 0.9 for GFI [193], CFI [194] and NFI [191]. Our model yields an adjusted Chi-square of 4.44, GFI of 0.96, CFI of 0.97, NFI of 0.96.

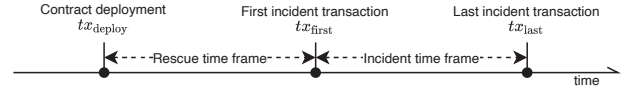


Fig. 6: An adversary $\mathbb{A}$ can deploy a smart contract with transaction tx_{deploy} and then initiate an incident by calling the contract with tx_{first} . Alternatively, the adversary may directly initiate the incident with tx_{first} in one of two ways: (i) without using a smart contract; or (ii) by batching the contract deployment and the initiation in a single transaction.

are relevant to Ethereum and BNB Smart Chain. (ii) We map the DeFiLama dataset with our incident dataset and find that 56 of the 776 protocols have been exploited before Apr 30, 2022. (iii) We construct a new audit dataset by taking snapshots and merging DeFiLama and DeFiYield audit databases [195] on June 20, 2022.

- **Result** According to our data, 23 of the 563 audited protocols (4.09%) have been attacked at least once, whereas 33 of the 213 non-audited protocols (15.49%) have been attacked. Hence, our data indicates that a security audit can decrease the average probability of an exploit by a factor of four. Due to the small sample size of only 56 matched incidents, our result can only be considered as a rough approximation.

V. INCIDENT DEFENSE

A. Rescue and Incident Time Frame

In the following, we investigate the rescue and the incident time frame (cf. Figure 6). The rescue time frame is the time between the adversarial contract deployment (tx_{deploy}) and the execution time of the first malicious state transition (tx_{first}). While the adversarial smart contract bytecode is already publicly available in the rescue time frame, the incident has not yet occurred. As such, defensive tools can theoretically reverse engineer the contract bytecode and determine its strategy using methods such as symbolic analysis, static analysis, and fuzzing, potentially mitigating or preventing harm. To our knowledge, no such just-in-time tool exists yet, which may explain why adversaries do not batch tx_{deploy} and tx_{first} into a single transaction yet (cf. Figure 7). The incident time frame, is the time that elapses between the execution of the first and last harmful state transition transactions. An $\mathbb{A}$ may prefer to keep the incident period as short as possible to maximize the attack’s success rate, which however may not always be possible due to gas constraints, protocol design, etc.

Figure 7 lays out the durations of the attack and rescue time frames. We discover that 103 (56%) attacks are not executed atomically, granting a rescue time frame for defenders. PRO layer incidents have the shortest average rescue time frame duration of $1h \pm 4.1$. The “Formation.Fi” incident has the longest rescue time frame, lasting approximately 25 days.

B. Bytecode Similarity Analysis

In the smart contracts ecosystem, code cloning has been utilized to measure the code similarity of deployed con-

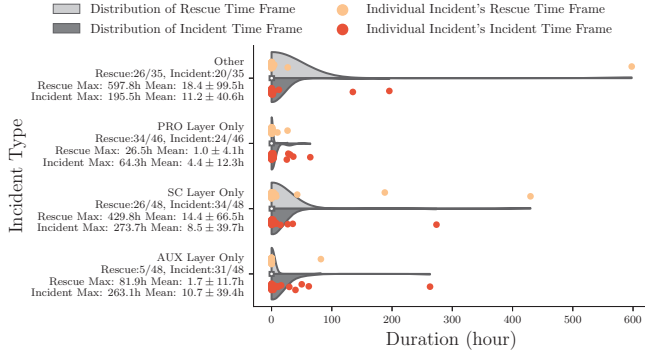


Fig. 7: The incident and rescue time frame per incident type. For example, we observe that 34 of the 46 PRO layer only incidents (74%) deploy smart contract(s) prior to the incident. The average rescue time frame for PRO layer is 1 ± 4.1 hours, with the longest rescue time frame being 26.5 hours.

Category	Similarity Threshold	Contracts			Unique Incidents		
		Total	Clusters	Detectable	Total	Clusters	Detectable
Vulnerable	100%	38	7	31	5	2	3
	80%	85	26	59	50	23	27
Adversarial	100%	29	6	23	0	0	0
	80%	73	23	50	31	13	18

TABLE VII: We perform bytecode similarity analysis on our incident dataset, which includes in total 173 vulnerable and 155 adversarial contracts. We identify 7 clusters of “exact match” vulnerable contracts (in total 38 vulnerable contracts), where contracts within the same cluster have a pairwise similarity score of 100%. Therefore, we infer that $38 - 7 = 31$ vulnerable contracts could be detected prior to the incident by comparing with previous known vulnerable contracts. Similarly, we infer that 23 adversarial contracts could be detected by comparing with previous known attacks.

tracts [196], identify plagiarized dApps [197], and vulnerability detection [198]. In this work, we employ code cloning to quantify bytecode similarity between all exploited DeFi protocols and adversarial contracts studied in this work. Note that we choose to perform our study at the deployed bytecode level as opposed to the source code level, because smart contract developers can close-source the contract code.

Methodology: Our code cloning detection method is inspired by the works of Kiffer et al. [196] and He et al. [197]. Specifically, to group similar smart contracts, we first identify and remove the Swarm code part from the bytecodes as it is not served for execution purposes. Then, we disassemble the bytecodes and remove the PUSH instructions’ arguments. Next, similar to [196], we compute hypervectors of n -grams ($n = 5$) of Ethereum opcodes for each contract. In order to compare two contracts, we compute the Jaccard similarity of their respective hypervectors. Finally, to cluster smart contracts into groups, we require a similarity score greater than 80% that the previous study suggests [196] [197].

Results: Table VII presents the results of the similarity anal-

ysis. We apply the above-mentioned methodology to cluster 173 vulnerable contracts and 155 adversarial contracts in our dataset. Using a similarity score threshold of 80%, we group vulnerable and adversarial smart contracts into 26 and 23 clusters, respectively. In addition, we note that in some clusters, all contracts are associated with a single incidence. To address more intriguing questions, such as how many comparable adversarial contracts attack different protocols (or different vulnerabilities in the same protocol), we restrict each cluster to a single contract per incident (c.f. Table VII).

We manually investigate the remaining clusters to acquire additional insights. For the vulnerable contracts, the clusters contain contracts that are part of DeFi protocols with similar functionalities (e.g., bridges and yield farming applications). Additionally, the exploitation of identical contracts is nearly equal (e.g., exploiting the same issue with equivalent transactions). In contrast, for similar vulnerable contracts, the exploits are not the same, but the incident cause is typically the same. For example, we identify two adversaries that exploit an issue on the same function in two smart contracts used as bridges, which fork the same smart contract. Specifically, although the implementation of the function is slightly different in the two contracts, both protocols introduce a vulnerability in the exact function while forking and modifying the same contract.

The most notable outcome of our similarity analysis is the identification of clusters of adversarial smart contracts that target distinct DeFi protocols with similar vulnerabilities (e.g., oracle manipulation). An analysis of historical blockchain data could reveal more adversarial smart contracts. Furthermore, we could potentially identify adversarial smart contracts in real-time, given that the time frame is long enough, by applying a more sophisticated similarity detection technique that could work on a more fine-grained level (e.g., function-level). Combining this with other program analysis techniques could potentially mitigate or prevent exploits (c.f. Section V-A).

Limitations: Our methodology cannot cluster similar contracts that employ different compilers and optimization choices. In addition, if an adversary choose to obfuscate the bytecode by, for example, injecting unused function code into the contract, our method becomes less effective. We therefore highlight the application of more sophisticate strategies as an interesting avenue for future work [199].

C. Front-Running as a Service (FaaS) Usage

FaaS are servers to which a trader’s transactions can be privately forwarded to miners that peer with the FaaS. We find that at least 18 incidents are executed through FaaS using Flashbots API on Ethereum. The first attack going through Flashbots happened on July 12, 2021.

- **Arbitrageurs Accelerate Attacks:** We manually examined each Flashbots bundle and discover that 6 of the 18 incidents appear to be accelerated by, e.g., arbitrage traders. We find that this is due to adversaries conducting incidents with sub-optimal strategy, resulting in extractable BEV opportunities. Trading bots will then compete for these BEV opportunities by back-running incident transactions with FaaS.

Algorithm 1: Source of Funds Tracing Algorithm

Input: Current highest block $b_{current}$; Tracing address T ; Starting block for post-incident tracing b_{post} ;
Transaction nonce equals the number of transaction sent; Algorithm OneHopPreIncidentTracing($T, b_{current}$):
 $b_{first} \leftarrow$ Binary search between block 0 and $b_{current}$ where T 's nonce equals 0 in b_{first} , and T 's nonce greater than 0 in $b_{first} + 1$.
 $b_{funding} \leftarrow$ Binary search between block 0 and b_{first} where T 's balance is greater than 0 in $b_{funding}$ and T 's balance equals 0 in $b_{funding} - 1$.
foreach $tx \in \{tx_{b_{funding}}^0, \dots\}$ **do**
 if Replay tx and finds native token transfer to T **then**
 return tx
 end
end
end

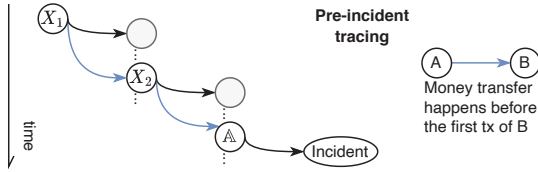


Fig. 8: Overview of the money tracing methodology. We start with the adversarial address (A), then iteratively determine the addresses that provide the initial source of funds (i.e., X_2 and X_1 , analogous to depth-first search).

- **Private Adversarial Transactions:** Adversaries can execute an incident using FaaS services, without broadcasting any transactions on the public blockchain P2P network. As a result, only entities with sequencer knowledge (K_2) are able to defend against these adversaries (e.g., perform bytecode similarity analysis) prior to transaction confirmation.

D. Money Tracing

Adversaries require a source of funds to issue transactions to execute incidents. A may attempt to break the linkability of their source of funds to evade potential legal ramifications. This section proposes a money tracing methodology to analyze the pre-incident flow of funds (cf. Figure 8).

An incident's source of funds is usually originating from a native coin transfer, e.g., from an address X to an address Y , i.e., $X \rightarrow Y$. We apply Algorithm 1 to identify the funding transaction $X \rightarrow Y$ for address Y . We abbreviate our notation with $X \xrightarrow{h} Y$, representing h hops transfer (i.e., $X \rightarrow I_1 \rightarrow \dots \rightarrow I_{h-1} \rightarrow Y$). To our knowledge, the current literature has not proposed any methodology to trace an incident's source of funds on an account-based ledger.

Pattern	Total	$h = 1$	$h = 2$	$h \geq 3$
Pre-incident (76 incidents in total, excluding U_2 -non-monetary adversaries)				
Centralized Exchange $\xrightarrow{h} A$	128(49.0%)	40(15.3%)	23(8.8%)	65(24.9%)
Tornado.Cash $\xrightarrow{h} A$	94(36.0%)	67(25.7%)	19(7.3%)	8(3.1%)
Typhoon.Network $\xrightarrow{h} A$	9(3.4%)	6(2.3%)	2(0.8%)	1(0.4%)
Mining Pool $\xrightarrow{h} A$	7(2.7%)	-	1(0.4%)	6(2.3%)
Cross-chain Bridge $\xrightarrow{h} A$	5(1.9%)	3(1.1%)	2(0.8%)	0(0.0%)
Unknown		18(6.9%)		

TABLE VIII: Source of funds identified for all 261 adversaries. h represents the number of hops (i.e. transactions) from the source of funds, e.g., In total, 73(28.0%) adversaries (92(50.8%) incidents) source the funds directly from a mixer.

- **Centralized Exchange:** We observe that 12(7.3%) (on Ethereum) and 21(8.0%) (on BSC) adversaries directly withdraw from exchange wallets ($h = 1$). The identities of these attackers can be revealed if the corresponding exchanges comply with Know Your Customer (KYC) requirements. For indirect exchange withdrawals ($h > 1$), we can only determine that A is linked to the withdrawer, but not whether the withdrawer is the attacker.
- **Mixer:** 55(21%) (on ETH) and 12(4.6%) (on BSC) adversaries receive their initial funds directly from a mixer ($h = 1$). Note that we classify a mixer as the source of funds only if a so-called relayer executes the withdrawal transaction (i.e., a third-party paying the transaction fees in the native blockchain coin); otherwise, we assume that the withdrawal fee payer is linked to the withdrawer and continue tracing the money flow. Relayers help to break address linkability, by paying the transaction fees (gas fee) of mixer withdrawal transactions in exchange for a commission on the withdrawal value.
- **Cross-chain Bridge:** Four attackers directly withdraw their source of funds from a blockchain bridge ($h = 1$).

Linked Incidents We discover that the adversarial address in 13 incidents can be linked to another incident's adversary within three hops (cf. Table X in the appendix).

Limitations We utilize Ether- and Bscscan²³ to identify the addresses of centralized exchanges and cross-chain bridges. Our dataset therefore inherits potential data completeness issues from Ether- and Bscscan.

VI. DISCUSSION

DeFi Incidents — Another Cat and Mouse Game: Analog to traditional information security, DeFi incidents can be perceived as a cat-and-mouse game, in which defenders attempt to minimize the security risk surface while attackers breach defenses. In the following, we extract insights on the current state of this contest, highlight key findings, discuss their implications and make recommendations for future research.

- 1) **Insight - Understudied NET and CON incidents:** We observe that NET and CON-related incidents are studied in 29% and 26% of academic papers (excluding tools, SoKs and surveys). However, only two tools (SquirRL [83],

²³<https://etherscan.io/labelcloud> and <https://bscscan.com/labelcloud>.

Layers	Surveys/SoKs		Tools		Papers		Audit reports		Incidents
	◆	◇	◆	◇	◆	◇	◆	◇	◆
Total	7		29		42		30		181
NET	4(57%)	19%	-	-	12(29%)	4%	-	-	4(2%)
CON	3(43%)	13%	2(7%)	2%	11(26%)	5%	-	-	0(0%)
SC	6(86%)	31%	26(90%)	20%	15(36%)	4%	29(97%)	35%	77(42%)
PRO	5(71%)	13%	15(52%)	6%	12(29%)	3%	19(63%)	14%	73(40%)
AUX	4(57%)	10%	2(7%)	1%	6(14%)	2%	14(47%)	5%	56(30%)

TABLE IX: Distributions of works under investigation according to the DeFi reference frame (cf. Section II-A). ◆ - the number and percentage of research items related to a system layer; ◇ - the average ratio of incident types each research item covers. For example, 15 of the 29 tools (52%) relate to PRO layer incidents, but each tool on average only covers 6% of the common PRO layer incident causes we identify.

DeFiPoser [9]) as well as 2% and 0% of the in-the-wild incidents relate to the NET and CON layers, respectively. While related works have surprisingly identified evidence of miner misbehavior in block header timestamps for financial gain [137], we note that: (a) it is not trivial to identify NET and CON incidents with absolute certainty (e.g., transaction censoring, selfish mining attack and block reorganization attack); and (b) to our knowledge, no publicly available tool can comprehensively detect potential NET and CON incidents in DeFi. As such, we suspect that more incidents have yet to be discovered. Furthermore, we notice that none of the industrial DeFi audit reports explicitly address potential NET and CON incidents, while some companies have previously performed NET and CON auditing for layer 1 and 2 blockchains²⁴.

- 2) **Challenge - Low coverage for PRO incidents:** SC and PRO layer incidents are the most common incident type (42% and 40%, respectively). Security tools, however, only cover 52% of the PRO layer incident types on average, which is less than SC layer (90%). As such, our dataset indicates that most defense tools still focus on SC vulnerabilities. The literature, however, suggests that the development of effective and generic PRO incident defense tools remains an open security challenge [9]. This is mainly due to DeFi's composability feature, which leads to action path explosion in detecting PRO layer vulnerabilities.
- 3) **Insight - Repeated on-chain oracle manipulation:** We discover 28 (15%) on-chain oracle manipulation incidents on Ethereum and BSC, which is the most common PRO layer incident type. On-chain oracle manipulation is one type of composability attack, which implies the adversary has C_{PRO}^3 capability. Repeated on-chain oracle manipulation indicates the need for tools to automatically identify such attack. To our knowledge, only DeFiRanger [97] and DeFiPoser [9] can detect oracle manipulation vulnerabilities. DeFiRanger can only identify observed attack transac-

tions, whereas DeFiPoser can identify new vulnerabilities in real-time, but necessitates manual and costly modeling of the captured DeFi protocols.

- 4) **Insight - Permissionless interactions are dangerous:** The permissionless interactions between various DeFi protocols can further broaden the attack surface. According to our dataset, in 19 (10.5%) incidents, adversaries utilize or deploy a contract (C_{PRO}^5), which complies with the accepted ABI interface, but contains incompatible implementation logic that causes harm²⁵. The underlying cause of these incidents is that the victims only constrain the contract function interface, not how the contract is implemented. We are, however, unaware of any viable way to efficiently verify code implementation on-chain due to the limitations of the current SC layer design. An alternative solution for constraining the contract with which a protocol or its user interacts is to implement a whitelist, where a DeFi protocol can only interact with other protocols in the whitelist.
- 5) **Insight - The identities of the attackers may still be revealed:** Although mixers are available on both Ethereum and BSC, our empirical result shows that only 38% of attackers obtain their source of funds from mixers (i.e., C_{PRO}^1). The majority of attackers interact with AUX services, such as centralized exchanges, and mining pools, which may provide stored personally identifiable information upon regulatory requests. Note that we naively assume mixers leaking the least side-channel information compared to other methodologies. Wang *et al.* [29] develop heuristics to reduce the anonymity set of Tornado.cash and Typhoon mixers on Ethereum and BSC. Quesnelle *et al.* [200] and Kappos *et al.* [201] investigate Zcash and show that the anonymity set size can be significantly reduced using simple heuristics to link transactions. Tran *et al.* [202] and Pakki *et al.* [203] show that existing mixer services are vulnerable to various threats such as permutation leak.
- 6) **Insight - Adversaries can be front-run during the rescue time frame:** Su *et al.* [99] discover that blockchain adversaries test their code by sending several transactions to the victim protocol before the actual attack. We initially questioned this finding because anyone can inspect the adversarial smart contract bytecode and transactions on the P2P layer, and therefore can front-run the adversaries to rescue the victim protocol. The optimal strategy for $\mathbb{A}$ is to emulate the state transitions off-chain, then deploy and exploit in one single transaction (i.e., the capability C_{PRO}^4). Surprisingly, our empirical results support Su *et al.* [99] (cf. Section V-A). We encourage the development of tools to front-run adversaries during this rescue time frame.
- 7) **Challenge - Absence of intrusion detection tools:** Only one incident in our dataset has triggered the emergency pause within the first hour of the incident. This indicates the absence of intrusion detection tools to automatically trigger emergency pauses. We anticipate that just-in-time detection

²⁴TrailOfBits for example audits many L1 and L2 blockchain projects, such as Arbitrum, THORChain, ZCash, etc. (<https://github.com/trailofbits/publications#blockchain-protocols-and-software>)

²⁵i.e. the following incident types: (i) token standard incompatibility; (ii) camouflage a token contract or (iii) camouflage a non-token contract

of abnormal protocol states or malicious transactions will receive increased attention in future studies.

- 8) **Insight - Adversarial and vulnerable contracts are detectable:** We show that SoTA similarity analysis can detect vulnerable and adversarial contracts. For instance, we identify 31/23 exactly matching vulnerable/adversarial contracts (i.e., bytecode similarity score of 100%) when compared to previously known incidents.

VII. RELATED WORKS

Cyber Risks: Sheyner *et al.* [204] outline an algorithm that can automatically generate attack graphs and analyze network security. Wang *et al.* [205] present a framework for measuring various aspects of network security metrics based on attack graphs. Khan *et al.* [206] propose a generalized mathematical model for cybersecurity that quantifies a set of parameters including risk, vulnerability, threat, attack, consequence, and reliability. Amin *et al.* [207] adopt the structural Bayesian Network to capture the relationship between financial loss, cyber risk and resilience, as well as developed a scorecard based approach to qualitatively assess the level of cyber risk. We refer interested readers to an SoK that thoroughly categorizes previous cyber risk studies [186]. While the research literature of cyber risks span over 30 years, DeFi is a relatively recent area with fewer works (cf. Table III).

DeFi Security: This paper proposes a five-layer system model as well as a comprehensive taxonomy of threat models that are used to measure and compare DeFi incidents. In the following, we present an overview of the most recent DeFi related survey and SoK papers, while highlighting the differences to contrast our work. Praitheeshan *et al.* [10] identify 19 software security issues and 16 Ethereum smart contract vulnerabilities, with 14 of them on smart contract layer. Homoliak *et al.* [11] present a stacked security model with four layers and systemized the vulnerabilities, threats, and countermeasures for each layer. Unfortunately, this research is not able to cover any smart contract layer vulnerabilities. Saad *et al.* [12] categorize 22 attack vectors in terms of its vulnerability origins (i.e., blockchain structure, P2P system and blockchain applications) and analyze the entities (e.g., miners, mining pools, users, exchanges, etc.) involved in each types of attacks. However, their examinations on protocol layer vulnerabilities and third-party vulnerabilities are conspicuously inadequate. Chen *et al.* [13] provide a comprehensive systematization of vulnerabilities, attacks, and defenses on four blockchain layers with detailed discussion on the relationships between them. Despite being able to cover in total of 40 vulnerabilities, this study does not state any vulnerabilities that are related to DeFi composability. Werner *et al.* [14] present a systematization of DeFi protocols and dissected DeFi related vulnerabilities with respect to technical and economic security. Nonetheless, this study lacks in-depth analysis of consensus and network layer vulnerabilities and does not provide generic measures to quantify the harm of DeFi incidents. Atzei *et al.* [15] investigate the security vulnerability on Ethereum and provided a taxonomy of the common programming pitfalls. Nevertheless,

the vulnerability coverage of this work is unsatisfactory as it exclusively focuses on smart contract layer. Samree *et al.* [16] identify 8 application level security vulnerabilities on the smart contract layer, analyze past attack incidents and categorize detection tools. However, this study also focuses on addressing smart contract vulnerabilities. Wan *et al.* [121] conduct 13 interviews and 156 surveys to investigate the practitioners' perceptions and practices on smart contract security. They, however, do not reveal how much effort was allocated into the security of each system layer. For studies and tools related to specific incidents, we refer interested readers to Table III.

Code Cloning: Code clone detection has been extensively explored in the literature for both source code [208] and binary programs [209]. Token based [210], tree based [211], graph based [212], text based [213], and deep learning based [214] techniques are the most prevalent techniques explored for code cloning. Applications of code cloning include bug detection, malware detection, patch analysis, plagiarism detection, and code similarity [208], [209], [215], [216]. Smart contract code cloning has been utilized primarily for computing duplication [196]–[198], [217]–[220] and vulnerability search [198], [217]. In this work, we apply a code cloning detection for comparing vulnerable and adversarial smart contracts.

Blockchain money tracing and account linking: Androulaki *et al.* [221] evaluate the privacy provisions in Bitcoin and show that nearly 40% of user profiles can be recovered. Meiklejohn *et al.* [222] apply heuristic clustering to group Bitcoin wallets. Yousaf *et al.* [223] develop heuristics allowing to trace transactions across blockchains. Victor [224] proposes heuristics to cluster Ethereum addresses by analyzing the phenomena surrounding deposit addresses, multiple participation in airdrops and token transfer authorization on Ethereum. The most relevant paper to this study is Su *et al.* [99], which analyze adversarial footprints and operational intents on Ethereum. In this work, we examine adversarial money flow before the attack to determine the source of funds.

VIII. CONCLUSION

This paper constructs a DeFi reference frame that categorizes 77 academic papers, 30 audit reports, and 181 incidents, which reveals the differences in how academia and the practitioners' community defend and inspect incidents. We investigate potential defense mechanisms, such as comparing victim/adversarial smart contract bytecodes, quantifying attack time frames, and tracing each attacker's source of funds. Our results suggest that DeFi security is still in its nascent stage, with many potential defense mechanisms requiring further research and implementation.

IX. ACKNOWLEDGEMENT

This work is partially supported by Chainlink labs, Swiss-Borg SA, NimiQ Foundation, Algorand, Lucerne University of Applied Sciences and Arts Switzerland, and the Federal Ministry of Education and Research of Germany ²⁶.

²⁶The programme of "Souverän. Digital. Vernetzt.". Joint project 6G-life, project identification number: 16KISK002

LINKED ADVERSARIES

Table X shows the linked adversaries based on source of fund tracing. We have identified six clusters, where the adversaries in five of the clusters are linked with three hops.

Suspects (A*)	Pattern	Incident	Date
0x8641df2D7C730A8A24db86693fc39F7A74Dd4e9D	A* $\xrightarrow{2}$ A	WildCredit	May 27, 2021
	A* $\xrightarrow{2}$ A	DeFiSaver	Oct 08, 2020
	A* $\xrightarrow{2}$ A	DODO	Mar 08, 2021
	A* $\xrightarrow{2}$ A	VisorFinance	Nov 26, 2021
	A* $\xrightarrow{2}$ A	MakerDAO	Mar 12, 2020
0x5b1839B202b67Db64e402a1501cf4f52f5eff03c	A* $\xrightarrow{3}$ A	BuccaneerFi	Mar 27, 2020
	A* $\xrightarrow{1}$ A	InfinityToken	Jan 26, 2022
0xC1A065a2d29995692735c82d228B63Df1732030E	A* $\xrightarrow{2}$ A	SodaFinance	Sep 20, 2020
	A* $\xrightarrow{1}$ A	BuccaneerFi	Aug 24, 2020
0xE4b3dD9839ed1780351Dc5412925cf05F07A1939	A* $\xrightarrow{2}$ A	bZx	Sep 13, 2020
	A* $\xrightarrow{2}$ A	ForceDAO	Apr 04, 2021
0x6bE5A267B04E9f24CdC1824fd38d63c436be91aB	A* $\xrightarrow{2}$ A	PancakeHunny	Jun 03, 2021
	A* $\xrightarrow{1}$ A	BoggedFinance	May 22, 2021
0x22B84d5FFeA8b801C0422AFe75237fA64A738c2	A* $\xrightarrow{8}$ A	MakerDAO	Mar 12, 2020
	A* $\xrightarrow{9}$ A	BadgerDAO	Nov 21, 2021

TABLE X: Linked adversaries based on pre-incident trace.

APPENDIX B

CUMULATIVE ABNORMAL RETURN (CAR)

We derive CAR with the following three steps:

- 1) Equation 3 fits β coefficient with the ordinary least square, where $R_{i,t}$, $R_{mkt,t}$, r_{f_t} denotes the token price, market price and risk-free rate²⁷ at tick $t \in [T_s - 144, T_s)$ respectively, α_i is the constant, and $\epsilon_{i,t}$ is the error term.

$$R_{i,t} - r_{f_t} = \alpha_i + \beta_i \cdot (R_{mkt,t} - r_{f_t}) + \epsilon_{i,t} \quad (3)$$

- 2) Calculate the ARs for each tick in the event timeframe using Equation 4, where $\hat{\beta}_i$ is the fitted β coefficient, $\mathbb{E}[R_{i,t}]$ is the expected return of token i .

$$AR_{i,t} = R_{i,t} - \mathbb{E}[R_{i,t}] = R_{i,t} - (\alpha_i + \hat{\beta}_i(R_{mkt,t} - r_{f_t}) + r_{f_t}) \quad (4)$$

- 3) Report the minimal CAR in Equation 5 to capture the price change pattern within the appearance of an anomaly.

$$CAR_i = \min_t [\sum_{t' < t} AR_{i,t'}] \quad (5)$$

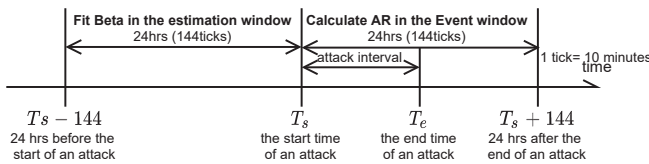


Fig. 9: Calculation of the cumulative abnormal return (CAR).

²⁷Typically, the 1- or 3-month US treasury bill yield is used as a proxy for r_{f_t} . However, due to unavailable high-frequency yield data, we assume $r_{f_t} = 0$. Token prices are obtained from various on-chain smart contracts, and the average price of Bitcoin and Ethereum is used as a market price proxy.

REFERENCES

- [1] DeFillama. (2022) DeFillama Dashboard. <https://defillama.com/>.
- [2] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, “Uniswap v3 core,” 2021.
- [3] M. Egorov, “Stableswap-efficient mechanism for stablecoin liquidity,” *Retrieved Feb.*, vol. 24, p. 2021, 2019.
- [4] J. A. Berg, R. Fritsch, L. Heimbach, and R. Wattenhofer, “An Empirical Study of Market Inefficiencies in Uniswap and SushiSwap,” in *2nd Workshop on Decentralized Finance (DeFi)*, Grenada, May 2022.
- [5] “Aave,” <https://github.com/aave/aave-protocol>, 2020.
- [6] “Compound finance,” <https://compound.finance/>, 2019.
- [7] “Synthetix,” <https://www.synthetix.io/>, 2020.
- [8] K. Qin, L. Zhou, B. Livshits, and A. Gervais, “Attacking the defi ecosystem with flash loans for fun and profit,” in *International Conference on Financial Cryptography and Data Security*. Springer, 2021, pp. 3–32.
- [9] L. Zhou, K. Qin, A. Cully, B. Livshits, and A. Gervais, “On the just-in-time discovery of profit-generating transactions in defi protocols,” *arXiv preprint arXiv:2103.02228*, 2021.
- [10] P. Praitheshan, L. Pan, J. Yu, J. Liu, and R. Doss, “Security analysis methods on ethereum smart contract vulnerabilities: a survey,” *arXiv preprint arXiv:1908.08605*, 2019.
- [11] I. Homoliak, S. Venugopalan, D. Reijbergen, Q. Hum, R. Schumi, and P. Szalachowski, “The security reference architecture for blockchains: Toward a standardized model for studying vulnerabilities, threats, and defenses,” *IEEE Communications Surveys & Tutorials*, vol. 23, 2020.
- [12] M. Saad, J. Spaulding, L. Njilla, C. Kamhoua, S. Shetty, D. Nyang, and A. Mohaisen, “Exploring the attack surface of blockchain: A systematic overview,” *arXiv preprint arXiv:1904.03487*, 2019.
- [13] H. Chen, M. Pendleton, L. Njilla, and S. Xu, “A survey on ethereum systems security: Vulnerabilities, attacks, and defenses,” *ACM Computing Surveys (CSUR)*, vol. 53, no. 3, pp. 1–43, 2020.
- [14] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt, “Sok: Decentralized finance (defi),” *arXiv preprint arXiv:2101.08778*, 2021.
- [15] N. Atzei, M. Bartoletti, and T. Cimoli, “A survey of attacks on ethereum smart contracts,” in *International conference on principles of security and trust*. Springer, 2017, pp. 164–186.
- [16] N. F. Samreen and M. H. Alalafi, “A survey of security vulnerabilities in ethereum smart contracts,” *arXiv preprint arXiv:2105.06974*, 2021.
- [17] S. Chaliasos, M. A. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, and B. Livshits, “Smart contract and defi security: Insights from tool evaluations and practitioner surveys,” 2023.
- [18] K. Qin, S. Chaliasos, L. Zhou, B. Livshits, D. Song, and A. Gervais, “The blockchain imitation game,” in *32th {USENIX} Security Symposium ({USENIX} Security 23)*.
- [19] S. Nakamoto and A. Bitcoin, “A peer-to-peer electronic cash system,” *Bitcoin*.—URL: <https://bitcoin.org/bitcoin.pdf>, vol. 4, 2008.
- [20] G. Wood *et al.*, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum project yellow paper*, vol. 151, 2014.
- [21] J. Xu, K. Paruch, S. Cousaert, and Y. Feng, “Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols,” *ACM Computing Surveys*, vol. 55, no. 11, pp. 1–50, 2023.
- [22] J. Xu and N. Vadgama, “From banks to defi: the evolution of the lending market,” *Enabling the Internet of Value: How Blockchain Connects Global Businesses*, pp. 53–66, 2022.
- [23] K. Qin, J. Ernstberger, L. Zhou, P. Jovanovic, and A. Gervais, “Mitigating decentralized finance liquidations with reversible call options,” *arXiv preprint arXiv:2303.15162*, 2023.
- [24] L. Heimbach, E. G. Schertenleib, and R. Wattenhofer, “Short squeeze in defi lending market: Decentralization in jeopardy?” *arXiv preprint arXiv:2302.04068*, 2023.
- [25] L. Heimbach, E. Schertenleib, and R. Wattenhofer, “Defi lending during the merge,” *arXiv preprint arXiv:2303.08748*, 2023.
- [26] A. Klages-Mundt, D. Harz, L. Gudgeon, J.-Y. Liu, and A. Minca, “Stablecoins 2.0: Economic foundations and risk-based models,” in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, 2020, pp. 59–79.
- [27] S. N. Steve Ellis, Ari Juels, “Chainlink: A decentralized oracle network,” 2017.
- [28] T. Mackinga, T. Nadahalli, and R. Wattenhofer, “TWAP Oracle Attacks: Easier Done than Said?” in *4th IEEE International Conference*

- on Blockchain and Cryptocurrency (ICBC), Virtual Conference, May 2022.
- [29] Z. Wang, S. Chaliasos, K. Qin, L. Zhou, L. Gao, P. Berrang, B. Livshits, and A. Gervais, "On how zero-knowledge proof blockchain mixers improve, and worsen user privacy," *arXiv preprint arXiv:2201.09035*, 2022.
 - [30] J. Xu and Y. Feng, "Reap the harvest on blockchain: A survey of yield farming protocols," *IEEE Transactions on Network and Service Management*, 2022.
 - [31] S. Cousaert, N. Vadgama, and J. Xu, "Token-based insurance solutions on blockchain," in *Blockchains and the Token Economy: Theory and Practice*. Springer, 2022, pp. 237–260.
 - [32] J. Xu, D. Perez, Y. Feng, and B. Livshits, "Auto. gov: Learning-based on-chain governance for decentralized finance (defi)," *arXiv preprint arXiv:2302.09551*, 2023.
 - [33] "Zcash," <https://z.cash>.
 - [34] A. Hinteregger and B. Haslhofer, "An Empirical Analysis of Monero Cross-Chain Traceability," *CoRR*, vol. abs/1812.02808, 2018. [Online]. Available: <http://arxiv.org/abs/1812.02808>
 - [35] S.-F. Sun, M. H. Au, J. K. Liu, and T. H. Yuen, "Ringct 2.0: A compact accumulator-based (linkable ring signature) protocol for blockchain cryptocurrency monero," in *European Symposium on Research in Computer Security*. Springer, 2017, pp. 456–474.
 - [36] D. V. Le and A. Gervais, "Amr: Autonomous coin mixer with privacy preserving reward distribution," *ACM Advances in Financial Technologies, AFT*, 2021.
 - [37] K. Qin, L. Zhou, Y. Afonin, L. Lazzaletti, and A. Gervais, "Cefi vs. defi—comparing centralized to decentralized finance," *arXiv preprint arXiv:2106.08157*, 2021.
 - [38] R. Braden, "Rfc1122: Requirements for internet hosts-communication layers," 1989.
 - [39] S. K. Kim, Z. Ma, S. Murali, J. Mason, A. Miller, and M. Bailey, "Measuring Ethereum network peers," in *Proceedings of the Internet Measurement Conference 2018*. ACM, 2018, pp. 91–104.
 - [40] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020.
 - [41] K. Qin, L. Zhou, and A. Gervais, "Quantifying blockchain extractable value: How dark is the forest?" in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022.
 - [42] Y. Wang, Y. Chen, H. Wu, L. Zhou, S. Deng, and R. Wattenhofer, "Cyclic Arbitrage in Decentralized Exchanges," in *The Web Conference 2022 (WWW)*, Lyon, France, April 2022.
 - [43] Y. Wang, P. Züst, Y. Yao, Z. Lu, and R. Wattenhofer, "Impact and User Perception of Sandwich Attacks in the DeFi Ecosystem," in *ACM CHI Conference on Human Factors in Computing Systems (CHI)*, New Orleans, LA, USA, May 2022.
 - [44] L. Heimbach and R. Wattenhofer, "Eliminating Sandwich Attacks with the Help of Game Theory," in *ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, Nagasaki, Japan, June 2022.
 - [45] Q. Wang, J. Yu, S. Chen, and Y. Xiang, "Sok: Diving into dag-based blockchain systems," *arXiv preprint arXiv:2012.06128*, 2020.
 - [46] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2016, pp. 17–30.
 - [47] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2018, pp. 931–948.
 - [48] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 583–598.
 - [49] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proceedings of the 2019 international conference on management of data*, 2019, pp. 123–140.
 - [50] L. Heimbach and R. Wattenhofer, "SoK: Preventing Transaction Re-ordering Manipulations in Decentralized Finance," in *4th ACM Conference on Advances in Financial Technologies (AFT)*, Cambridge, Massachusetts, USA, September 2022.
 - [51] R. Khalil, A. Gervais, and G. Felley, "NOCUST—A Securely Scalable Commit-Chain," 2018.
 - [52] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 1353–1370.
 - [53] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais, "Sok: Layer-two blockchain protocols," in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 201–226.
 - [54] F. Vogelsteller and V. Buterin, "Eip-20: Erc-20 token standard," 2015, <https://eips.ethereum.org/EIPS/eip-20>.
 - [55] E. Kuo, B. Iles, and M. R. Cruz, "Ampleforth: A new synthetic commodity," 2019.
 - [56] J. Dafflon, J. Baylina, and T. Shababi, "Eip-777: Erc777 token standard," 2017, <https://eips.ethereum.org/EIPS/eip-777>.
 - [57] "Home - alpha finance lab," <https://alphafinance.io>.
 - [58] "Harvest finance," <https://harvest.finance/>.
 - [59] L. Heimbach, Y. Wang, and R. Wattenhofer, "Behavior of Liquidity Providers in Decentralized Exchanges," in *2021 Crypto Valley Conference on Blockchain Technology (CVCBT)*, Rotkreuz, Switzerland, October 2021.
 - [60] L. Heimbach, E. Schertenleib, and R. Wattenhofer, "Risks and Returns of Uniswap V3 Liquidity Providers," in *4th ACM Conference on Advances in Financial Technologies (AFT)*, Cambridge, Massachusetts, USA, September 2022.
 - [61] A. Gervais, H. Ritzdorf, G. O. Karame, and S. Capkun, "Tampering with the delivery of blocks and transactions in bitcoin," in *Conference on Computer and Communications Security*. ACM, 2015.
 - [62] I. Eyal and E. G. Sirer, "Majority is not enough: Bitcoin mining is vulnerable," in *Financial Cryptography and Data Security*. Springer, 2014, pp. 436–454.
 - [63] F. Schär, "Decentralized finance: On blockchain-and smart contract-based financial markets," *FRB of St. Louis Review*, 2021.
 - [64] "CoinGecko yield farming survey 2020," <https://www.coingecko.com/buzz/yield-farming-survey-2020>.
 - [65] Y. Zhou, D. Kumar, S. Bakshi, J. Mason, A. Miller, and M. Bailey, "Erays: reverse engineering ethereum's opaque smart contracts," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 1371–1385.
 - [66] M. Rodler, W. Li, G. O. Karame, and L. Davi, "Evmpatch: timely and automated patching of ethereum smart contracts," in *30th {USENIX} Security Symposium ({USENIX} Security 21)*, 2021.
 - [67] J. Frank, C. Aschermann, and T. Holz, "{ETHBMC}: A bounded model checker for smart contracts," in *29th {USENIX} Security Symposium ({USENIX} Security 20)*, 2020, pp. 2757–2774.
 - [68] L. Breidenbach, P. Daian, F. Tramèr, and A. Juels, "Enter the hydra: Towards principled bug bounties and exploit-resistant smart contracts," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 1335–1352.
 - [69] G. A. Akerlof and J. L. Yellen, "A near-rational model of the business cycle, with wage and price inertia," *The Quarterly Journal of Economics*, vol. 100, pp. 823–838, 1985.
 - [70] N. Strong, "Modelling abnormal returns: A review article," *Journal of Business Finance & Accounting*, vol. 19, no. 4, pp. 533–553, 1992.
 - [71] S. So, S. Hong, and H. Oh, "Smartest: Effectively hunting vulnerable transaction sequences in smart contracts through language model-guided symbolic execution," in *30th {USENIX} Security Symposium ({USENIX} Security 21)*, 2021.
 - [72] M. Zhang, X. Zhang, Y. Zhang, and Z. Lin, "{TXSPECTOR}: Uncovering attacks in ethereum from transactions," in *29th {USENIX} Security Symposium ({USENIX} Security 20)*, 2020, pp. 2775–2792.
 - [73] S. Zhou, M. Möser, Z. Yang, B. Adida, T. Holz, J. Xiang, S. Goldfeder, Y. Cao, M. Plattner, X. Qin *et al.*, "An ever-evolving game: Evaluation of real-world attacks and defenses in ethereum ecosystem," in *29th {USENIX} Security Symposium ({USENIX} Security 20)*, 2020, pp. 2793–2810.
 - [74] J. Krupp and C. Rossow, "teether: Gnawing at ethereum to automatically exploit smart contracts," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 1317–1333.
 - [75] P. Bose, D. Das, Y. Chen, Y. Feng, C. Kruegel, and G. Vigna, "Sailfish: Vetting smart contract state-inconsistency bugs in seconds," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022.
 - [76] T. D. Nguyen, L. H. Pham, and J. Sun, "Sguard: Smart contracts made vulnerability-free," 2021.

- [77] J. Stephens, K. Ferles, B. Mariano, S. Lahiri, and I. Dillig, "Smartpulse: Automated checking of temporal properties in smart contracts," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021.
- [78] A. Permenev, D. Dimitrov, P. Tsankov, D. Drachler-Cohen, and M. Vechev, "Verx: Safety verification of smart contracts," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020.
- [79] S. So, M. Lee, J. Park, H. Lee, and H. Oh, "Verismart: A highly precise safety verifier for ethereum smart contracts," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1678–1694.
- [80] C. Schneidewind, I. Grishchenko, M. Scherer, and M. Maffei, "ethor: Practical and provably sound static analysis of ethereum smart contracts," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 621–640.
- [81] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 531–548.
- [82] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 67–82.
- [83] C. Hou, M. Zhou, Y. Ji, P. Daian, F. Tramer, G. Fanti, and A. Juels, "Squirrel: Automating attack analysis on blockchain incentive mechanisms with deep reinforcement learning," in *NDSS*, 2021.
- [84] T. Chen, R. Cao, T. Li, X. Luo, G. Gu, Y. Zhang, Z. Liao, H. Zhu, G. Chen, Z. He *et al.*, "Soda: A generic online detection framework for smart contracts," in *NDSS*, 2020.
- [85] M. Rodler, W. Li, G. O. Karame, and L. Davi, "Sereum: Protecting existing smart contracts against re-entrancy attacks," 2019.
- [86] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, "Zeus: Analyzing safety of smart contracts," in *Ndss*, 2018, pp. 1–12.
- [87] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "sfuzz: An efficient adaptive fuzzer for solidity smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 778–788.
- [88] N. Grech, L. Brent, B. Scholz, and Y. Smaragdakis, "Gigahorse: thorough, declarative decompilation of smart contracts," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 1176–1186.
- [89] J. Choi, G. Grieco, D. Kim, A. Groce, S. Kim, and S. K. Cha, "Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses," in *The 36th IEEE/ACM International Conference on Automated Software Engineering*. IEEE/ACM, 2021.
- [90] Y. Feng, E. Torlak, and R. Bodik, "Summary-based symbolic evaluation for smart contracts," in *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2020, pp. 1141–1152.
- [91] B. Jiang, Y. Liu, and W. Chan, "Contractfuzzer: Fuzzing smart contracts for vulnerability detection," in *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2018, pp. 259–269.
- [92] H. Liu, C. Liu, W. Zhao, Y. Jiang, and J. Sun, "S-gram: towards semantic-aware security auditing for ethereum smart contracts," in *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2018, pp. 814–819.
- [93] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 254–269.
- [94] L. Brent, A. Jurisevic, M. Kong, E. Liu, F. Gauthier, V. Gramoli, R. Holz, and B. Scholz, "Vandal: A scalable security analysis framework for smart contracts," *arXiv preprint arXiv:1809.03981*, 2018.
- [95] E. Albert, P. Gordillo, B. Livshits, A. Rubio, and I. Sergey, "Ethere: A framework for high-level analysis of ethereum bytecode," in *International symposium on automated technology for verification and analysis*. Springer, 2018, pp. 513–520.
- [96] I. Nikolić, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, "Finding the greedy, prodigal, and suicidal contracts at scale," in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 653–663.
- [97] S. Wu, D. Wang, J. He, Y. Zhou, L. Wu, X. Yuan, Q. He, and K. Ren, "Defranger: Detecting price manipulation attacks on defi applications," *arXiv preprint arXiv:2104.15068*, 2021.
- [98] T. Chen, X. Li, X. Luo, and X. Zhang, "Under-optimized smart contracts devour your money," in *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2017, pp. 442–446.
- [99] L. Su, X. Shen, X. Du, X. Liao, X. Wang, L. Xing, and B. Liu, "Evil under the sun: Understanding and discovering attacks on ethereum decentralized applications," in *30th {USENIX} Security Symposium ({USENIX} Security 21)*, 2021.
- [100] D. Perez and B. Livshits, "Smart contract vulnerabilities: Vulnerable does not imply exploited," in *30th {USENIX} Security Symposium ({USENIX} Security 21)*, 2021.
- [101] C. F. Torres, R. Camino, and R. State, "Frontrunner jones and the raiders of the dark forest: An empirical study of frontrunning on the ethereum blockchain," *arXiv preprint arXiv:2102.03347*, 2021.
- [102] P. Szalachowski, D. Reijnders, I. Homoliak, and S. Sun, "Strongchain: Transparent and collaborative proof-of-work consensus," in *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019, pp. 819–836.
- [103] C. F. Torres, M. Steichen *et al.*, "The art of the scam: Demystifying honeypots in ethereum smart contracts," in *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019, pp. 1591–1607.
- [104] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais, "High-frequency trading on decentralized on-chain exchanges," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 428–445.
- [105] E. Cecchetti, S. Yao, H. Ni, and A. C. Myers, "Compositional security for reentrant applications," *arXiv preprint arXiv:2103.08577*, 2021.
- [106] J. Jiao, S. Kan, S.-W. Lin, D. Sanan, Y. Liu, and J. Sun, "Semantic understanding of smart contracts: Executable operational semantics of solidity," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1695–1712.
- [107] R. Zhang and B. Preneel, "Lay down the common metrics: Evaluating proof-of-work consensus protocols' security," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 175–192.
- [108] M. Saad, A. Anwar, S. Ravi, and D. Mohaisen, "Revisiting nakamoto consensus in asynchronous networks: A comprehensive analysis of bitcoin safety and chainquality," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 988–1005.
- [109] A. Lewis-Pye and T. Roughgarden, "How does blockchain security dictate blockchain implementation?" in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1006–1019.
- [110] P. Das, A. Erwig, S. Faust, J. Loss, and S. Riahi, "The exact security of bip32 wallets," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1020–1042.
- [111] K. Li, Y. Wang, and Y. Tang, "Deter: Denial of ethereum txpool services," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1645–1667.
- [112] M. Saad, S. Chen, and D. Mohaisen, "Syncattack: Double-spending in bitcoin without mining power," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1668–1685.
- [113] M. Mirkin, Y. Ji, J. Pang, A. Klages-Mundt, I. Eyal, and A. Juels, "Bdos: Blockchain denial-of-service," in *Proceedings of the 2020 ACM SIGSAC conference on Computer and Communications Security*, 2020, pp. 601–619.
- [114] T. Chen, Y. Zhang, Z. Li, X. Luo, T. Wang, R. Cao, X. Xiao, and X. Zhang, "Tokenscope: Automatically detecting inconsistent behaviors of cryptocurrency tokens in ethereum," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, 2019, pp. 1503–1520.
- [115] P. Das, S. Faust, and J. Loss, "A formal treatment of deterministic wallets," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 651–668.
- [116] I. Tsabary and I. Eyal, "The gap game," in *Proceedings of the 2018 ACM SIGSAC conference on Computer and Communications Security*, 2018, pp. 713–728.
- [117] L. Kiffer, R. Rajaraman, and A. Shelat, "A better method to analyze blockchain consistency," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 729–744.
- [118] K. Li, J. Chen, X. Liu, Y. Tang, X. Wang, and X. Luo, "As strong as its weakest link: How to break blockchain dapps at rpc service,"

- in *28th Annual Network and Distributed System Security Symposium, NDSS*, 2021, pp. 21–25.
- [119] G. Bissias and B. N. Levine, “Bobtail: Improved blockchain security with low-variance mining,” in *NDSS*, 2020.
- [120] D. Perez and B. Livshits, “Broken metre: Attacking resource metering in evm,” 2020.
- [121] Z. Wan, X. Xia, D. Lo, J. Chen, X. Luo, and X. Yang, “Smart contract security: a practitioners’ perspective,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1410–1422.
- [122] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, “Empirical review of automated analysis tools on 47,587 ethereum smart contracts,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 530–541.
- [123] S. Hwang and S. Ryu, “Gap between theory and practice: An empirical study of security patches in solidity,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 542–553.
- [124] L. Liu, L. Wei, W. Zhang, M. Wen, Y. Liu, and S.-C. Cheung, “Characterizing transaction-reverting statements in ethereum smart contracts,” *arXiv preprint arXiv:2108.10799*, 2021.
- [125] Y. Xue, M. Ma, Y. Lin, Y. Sui, J. Ye, and T. Peng, “Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts,” in *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2020, pp. 1029–1040.
- [126] S. Grossman, I. Abraham, G. Golan-Gueta, Y. Michalevsky, N. Rinetzkly, M. Sagiv, and Y. Zohar, “Online detection of effectively callback free objects with applications to smart contracts,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 1–28, 2017.
- [127] A. Li, J. A. Choi, and F. Long, “Securing smart contract with runtime validation,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2020, pp. 438–453.
- [128] L. Brent, N. Grech, S. Lagouvardos, B. Scholz, and Y. Smaragdakis, “Ethainter: A smart contract security analyzer for composite vulnerabilities,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2020, pp. 454–469.
- [129] S. M. Beillahi, G. Ciocarlie, M. Emmi, and C. Enea, “Behavioral simulation for smart contracts,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2020, pp. 470–486.
- [130] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, “On the security and performance of proof of work blockchains,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2016, pp. 3–16.
- [131] K. Wüst and A. Gervais, “Ethereum eclipse attacks,” ETH Zurich, Tech. Rep., 2016.
- [132] Y. Marcus, E. Heilman, and S. Goldberg, “Low-resource eclipse attacks on ethereum’s peer-to-peer network,” *IACR Cryptol. ePrint Arch.*, vol. 2018, p. 236, 2018.
- [133] R. Rahimian and J. Clark, “Tokenhook: Secure erc-20 smart contract,” *arXiv preprint arXiv:2107.02997*, 2021.
- [134] L. Gudgeon, D. Perez, D. Harz, A. Gervais, and B. Livshits, “The decentralized financial crisis: Attacking defi,” *arXiv preprint arXiv:2002.08099*, 2020.
- [135] J. Guarnizo and P. Szalachowski, “Pdfs: practical data feed service for smart contracts,” in *European Symposium on Research in Computer Security*. Springer, 2019, pp. 767–789.
- [136] L. Breidenbach, C. Cachin, B. Chan, A. Coventry, S. Ellis, A. Juels, F. Koushanfar, A. Miller, B. Magauran, D. Moroz *et al.*, “Chainlink 2.0: Next steps in the evolution of decentralized oracle networks,” 2021.
- [137] A. YAISH, G. STERN, and A. ZOHAR, “Uncle maker:(time) stamping out the competition in ethereum,” 2022.
- [138] “Wakaswap audit,” <https://waka-finance-2.gitbook.io/waka-finance/documentation/audit>, 2021, beosin.
- [139] “Sato audit,” https://sato.trade/Smart_contract_security_audit_report%E2%80%9494SATO.pdf, 2021, beosin.
- [140] “Pinecone audit,” https://safefiles.defiyield.info/safe/files/audit/pdf/REP_Pinecone_Finance_2021_09_28.pdf, 2021, beosin.
- [141] “Ctoken audit,” 2021, beosin.
- [142] “Beatsquare audit,” 2021, beosin.
- [143] “Rabbit.fi audit,” https://github.com/peckshield/publications/blob/master/audit_reports/PeckShield-Audit-Report-Rabbit-v1.0.pdf, 2021, peckshield.
- [144] “Hegic audit,” https://safefiles.defiyield.info/safe/files/audit/pdf/PeckShield_Audit_Report_Hegic_v1_0.pdf, 2021, peckshield.
- [145] “Deri v2 audit,” https://github.com/peckshield/publications/blob/693bdb69e3e3e422b4f7e1f3130d841e631b4dab/audit_reports/PeckShield-Audit-Report-DeriV2-v1.0.pdf, 2021, peckshield.
- [146] “Coin98 audit,” https://safefiles.defiyield.info/safe/files/audit/pdf/PeckShield_Audit_Report_COIN98_v1_0.pdf, 2021, peckshield.
- [147] “Angrymining audit,” https://github.com/peckshield/publications/blob/master/audit_reports/PeckShield-Audit-Report-AngryMining-v1.0rc.pdf, 2021, peckshield.
- [148] “Jswap audit,” <https://www.slowmist.com/en/security-audit-certificate.html?id=928799684ad96ef4ed4b0c0fb12a5fae085456f874b19dc4300195b32a5a1431>, 2021, slowMist.
- [149] “Supremex audit,” <https://www.slowmist.com/security-audit-certificate.html?id=769a2454892441cfc9730e3fc39db48b75e9bb05ad3527ce1736342ff8ea8e3>, 2021, slowMist.
- [150] “Solyard audit,” <https://www.slowmist.com/security-audit-certificate.html?id=53e38102e25c3c6d8a8136edc7e859fde08ed93189c1535d642bb1cd656e5815>, 2021, slowMist.
- [151] “Cook finance audit,” <https://github.com/slowmist/Knowledge-Base/blob/master/open-report/SlowMist%20Audit%20Report%20-%20Cook%20Finance.pdf>, 2021, slowMist.
- [152] “Defi saver audit,” <https://github.com/defisaver/defisaver-v3-contracts/blob/main/audits/Consensus-Mar-2021.pdf>, 2021, consensus.
- [153] “Fei tribechief audit,” <https://consensus.net/diligence/audits/2021/07/fei-tribechief/>, 2021, consensus.
- [154] “Gitcoin audit,” <https://consensus.net/diligence/audits/2021/04/gitcoin-token-distribution/>, 2021, consensus.
- [155] “Wheat audit,” <https://consensus.net/diligence/audits/2021/06/growthd-efi-wheat/>, 2021, consensus.
- [156] “Umbra audit,” <https://consensus.net/diligence/audits/2021/03/umbra-smart-contracts/>, 2021, consensus.
- [157] “Zoo audit,” <https://www.certik.com/projects/zoocrypto>, 2021, certik.
- [158] “Trister’s lend audit,” <https://www.certik.com/projects/tristerlend>, 2021, certik.
- [159] “Rezerve audit,” <https://www.certik.com/projects/rezerve>, 2021, certik.
- [160] “Lfw audit,” <https://www.certik.com/projects/legendfantasywar>, 2021, certik.
- [161] “gamedao audit,” <https://www.certik.com/projects/gamedao>, 2021, certik.
- [162] “Complifi audit,” <https://github.com/trailofbits/publications/blob/master/reviews/CompliFi.pdf>, 2021, trail of Bits.
- [163] “Frax finance audit,” <https://github.com/trailofbits/publications/blob/master/reviews/FraxFinance.pdf>, 2021, trail of Bits.
- [164] “YearnV2 audit,” <https://github.com/trailofbits/publications/blob/master/reviews/YearnV2Vaults.pdf>, 2021, trail of Bits.
- [165] “Alpha homora audit,” <https://blog.openzeppelin.com/alpha-homora-v2/>, 2021, open Zeppelin.
- [166] “Celo audit,” <https://blog.openzeppelin.com/celo-contracts-audit/>, 2021, open Zeppelin.
- [167] “Fei audit,” <https://blog.openzeppelin.com/fei-protocol-audit/>, 2021, open Zeppelin.
- [168] “Defi plus.” [Online]. Available: <https://defipulse.com/>
- [169] “Circuit breaker definition,” <https://www.investopedia.com/terms/c/circuitbreaker.asp>.
- [170] K. G. Jöreskog, “A general method for estimating a linear structural equation system,” *ETS Research Bulletin Series*, vol. 1970, no. 2, pp. i–41, 1970.
- [171] C. Fornell and D. F. Larcker, “Structural equation models with unobservable variables and measurement error: Algebra and statistics,” 1981.
- [172] K. G. Jöreskog and D. Sörbom, “Recent developments in structural equation modeling,” *Journal of marketing research*, vol. 19, no. 4, pp. 404–416, 1982.
- [173] Jöreskog, Karl G and Sörbom, Dag, *LISREL 8: Structural equation modeling with the SIMPLIS command language*. Scientific Software International, 1993.
- [174] R. H. Hoyle, *Structural equation modeling: Concepts, issues, and applications*. Sage, 1995.
- [175] Hoyle, Rick H, “The structural equation modeling approach: Basic concepts and fundamental issues.” 1995.

- [176] D. Gefen, D. Straub, and M.-C. Boudreau, "Structural equation modeling and regression: Guidelines for research practice," *Communications of the association for information systems*, vol. 4, no. 1, p. 7, 2000.
- [177] R. E. Schumacker and R. G. Lomax, *A beginner's guide to structural equation modeling*. psychology press, 2004.
- [178] J. B. Ullman and P. M. Bentler, "Structural equation modeling," *Handbook of Psychology, Second Edition*, vol. 2, 2012.
- [179] B. M. Byrne, *Structural equation modeling with Mplus: Basic concepts, applications, and programming*. routledge, 2013.
- [180] R. B. Kline, *Principles and practice of structural equation modeling*. Guilford publications, 2015.
- [181] D. Harrington, *Confirmatory factor analysis*. Oxford university press, 2009.
- [182] T. A. Brown and M. T. Moore, "Confirmatory factor analysis," *Handbook of structural equation modeling*, vol. 361, p. 379, 2012.
- [183] T. A. Brown, *Confirmatory factor analysis for applied research*. Guilford publications, 2015.
- [184] E. J. Wolf and T. A. Brown, "Structural equation modeling," in *The Oxford Handbook of Research Strategies for Clinical Psychology*, 1996.
- [185] G. Meshcheryakov, A. A. Igolkina, and M. G. Samsonova, "semopy 2: A structural equation modeling package with random effects in python," *arXiv preprint arXiv:2106.01140*, 2021.
- [186] D. W. Woods and R. Böhme, "Systematization of knowledge: Quantifying cyber risk," in *IEEE Symposium on Security & Privacy*, 2021.
- [187] B. T. West, K. B. Welch, and A. T. Galecki, *Linear mixed models: a practical guide using statistical software*. Chapman and Hall/CRC, 2006.
- [188] P. G. Freund and M. A. Rubin, "Dynamics of dimensional reduction," *Physics Letters B*, vol. 97, no. 2, pp. 233–235, 1980.
- [189] A. Satorra and P. M. Bentler, "Corrections to test statistics and standard errors in covariance structure analysis." 1994.
- [190] P. M. Bentler, "Comparative fit indexes in structural models," *Psychological bulletin*, vol. 107, no. 2, p. 238, 1990.
- [191] P. M. Bentler and D. G. Bonett, "Significance tests and goodness of fit in the analysis of covariance structures," *Psychological bulletin*, vol. 88, no. 3, p. 588, 1980.
- [192] B. Wheaton, B. Muthen, D. F. Alwin, and G. F. Summers, "Assessing reliability and stability in panel models," *Sociological methodology*, vol. 8, pp. 84–136, 1977.
- [193] J. Sun, "Assessing goodness of fit in confirmatory factor analysis," *Measurement and evaluation in counseling and development*, vol. 37, no. 4, pp. 240–256, 2005.
- [194] P. Barrett, "Structural equation modelling: Adjudging model fit," *Personality and Individual differences*, vol. 42, no. 5, pp. 815–824, 2007.
- [195] DeFiYield. (2022) DeFiYield. <https://de.fi/audit-database>.
- [196] L. Kiffer, D. Levin, and A. Mislove, "Analyzing ethereum's contract topology," in *Proceedings of the Internet Measurement Conference 2018*, 2018, pp. 494–499.
- [197] N. He, L. Wu, H. Wang, Y. Guo, and X. Jiang, "Characterizing code clones in the ethereum smart contract ecosystem," in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 654–675.
- [198] Z. Gao, L. Jiang, X. Xia, D. Lo, and J. Grundy, "Checking smart contracts with structural code embedding," *IEEE Transactions on Software Engineering*, 2020.
- [199] D. Zhu, J. Pang, X. Zhou, and W. Han, "Similarity measure for smart contract bytecode based on cfg feature extraction," in *2021 International Conference on Computer Information Science and Artificial Intelligence (CISAI)*. IEEE, 2021, pp. 558–562.
- [200] J. Quesnelle, "On the linkability of zcash transactions," *arXiv preprint arXiv:1712.01210*, 2017.
- [201] G. Kappos, H. Yousaf, M. Maller, and S. Meiklejohn, "An empirical analysis of anonymity in zcash," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 463–477.
- [202] M. Tran, L. Luu, M. S. Kang, I. Bentov, and P. Saxena, "Obscuro: A bitcoin mixer using trusted execution environments," in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 692–701.
- [203] J. Pakki, Y. Shoshitaishvili, R. Wang, T. Bao, and A. Doupé, "Everything you ever wanted to know about bitcoin mixers (but were afraid to ask)," in *International Conference on Financial Cryptography and Data Security*. Springer, 2021, pp. 117–146.
- [204] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proceedings 2002 IEEE Symposium on Security and Privacy*. IEEE, 2002, pp. 273–284.
- [205] L. Wang, A. Singhal, and S. Jajodia, "Toward measuring network security using attack graphs," in *Proceedings of the 2007 ACM workshop on Quality of protection*, 2007.
- [206] M. A. Khan and M. Hussain, "Cyber security quantification model," in *Proceedings of the 3rd international conference on Security of information and networks*, 2010, pp. 142–148.
- [207] Z. Amin, "A practical road map for assessing cyber risk," *Journal of Risk Research*, vol. 22, no. 1, pp. 32–43, 2019.
- [208] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," *Science of computer programming*, vol. 74, no. 7, pp. 470–495, 2009.
- [209] I. U. Haq and J. Caballero, "A survey of binary code similarity," *ACM Computing Surveys (CSUR)*, vol. 54, no. 3, pp. 1–38, 2021.
- [210] B. S. Baker, "On finding duplication and near-duplication in large software systems," in *Proceedings of 2nd Working Conference on Reverse Engineering*. IEEE, 1995, pp. 86–95.
- [211] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier, "Clone detection using abstract syntax trees," in *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*. IEEE, 1998, pp. 368–377.
- [212] R. Komondoor and S. Horwitz, "Using slicing to identify duplication in source code," in *International static analysis symposium*. Springer, 2001, pp. 40–56.
- [213] S. Ducasse, M. Rieger, and S. Demeyer, "A language independent approach for detecting duplicated code," in *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99). Software Maintenance for Business Change' (Cat. No. 99CB36360)*. IEEE, 1999, pp. 109–118.
- [214] M. White, M. Tufano, C. Vendome, and D. Poshyanyk, "Deep learning code fragments for code clone detection," in *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2016, pp. 87–98.
- [215] M. Novak, M. Joy, and D. Kermek, "Source-code similarity detection and detection tools used in academia: a systematic review," *ACM Transactions on Computing Education (TOCE)*, vol. 19, no. 3, pp. 1–37, 2019.
- [216] C. K. Roy and J. R. Cordy, "A survey on software clone detection research," *Queen's School of computing TR*, vol. 541, no. 115, pp. 64–68, 2007.
- [217] H. Liu, Z. Yang, Y. Jiang, W. Zhao, and J. Sun, "Enabling clone detection for ethereum via smart contract birthmarks," in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 2019, pp. 105–115.
- [218] X. Chen, P. Liao, Y. Zhang, Y. Huang, and Z. Zheng, "Understanding code reuse in smart contracts," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2021, pp. 470–479.
- [219] M. Kondo, G. A. Oliva, Z. M. J. Jiang, A. E. Hassan, and O. Mizuno, "Code cloning in smart contracts: a case study on verified contracts from the ethereum blockchain platform," *Empirical Software Engineering*, vol. 25, no. 6, pp. 4617–4675, 2020.
- [220] D. Zhu, F. Yue, J. Pang, X. Zhou, W. Han, and F. Liu, "Bytecode similarity detection of smart contract across optimization options and compiler versions based on triplet network," *Electronics*, vol. 11, no. 4, p. 597, 2022.
- [221] E. Androulaki, G. O. Karame, M. Roeschlin, T. Scherer, and S. Capkun, "Evaluating user privacy in bitcoin," in *International Conference on Financial Cryptography and Data Security*. Springer, 2013, pp. 34–51.
- [222] S. Meiklejohn, M. Pomarole, G. Jordan, K. Levchenko, D. McCoy, G. M. Voelker, and S. Savage, "A fistful of bitcoins: characterizing payments among men with no names," in *Proceedings of the 2013 conference on Internet measurement conference*. ACM, 2013, pp. 127–140.
- [223] H. Yousaf, G. Kappos, and S. Meiklejohn, "Tracing transactions across cryptocurrency ledgers," in *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019, pp. 837–850.
- [224] F. Victor, "Address clustering heuristics for ethereum," in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 617–633.